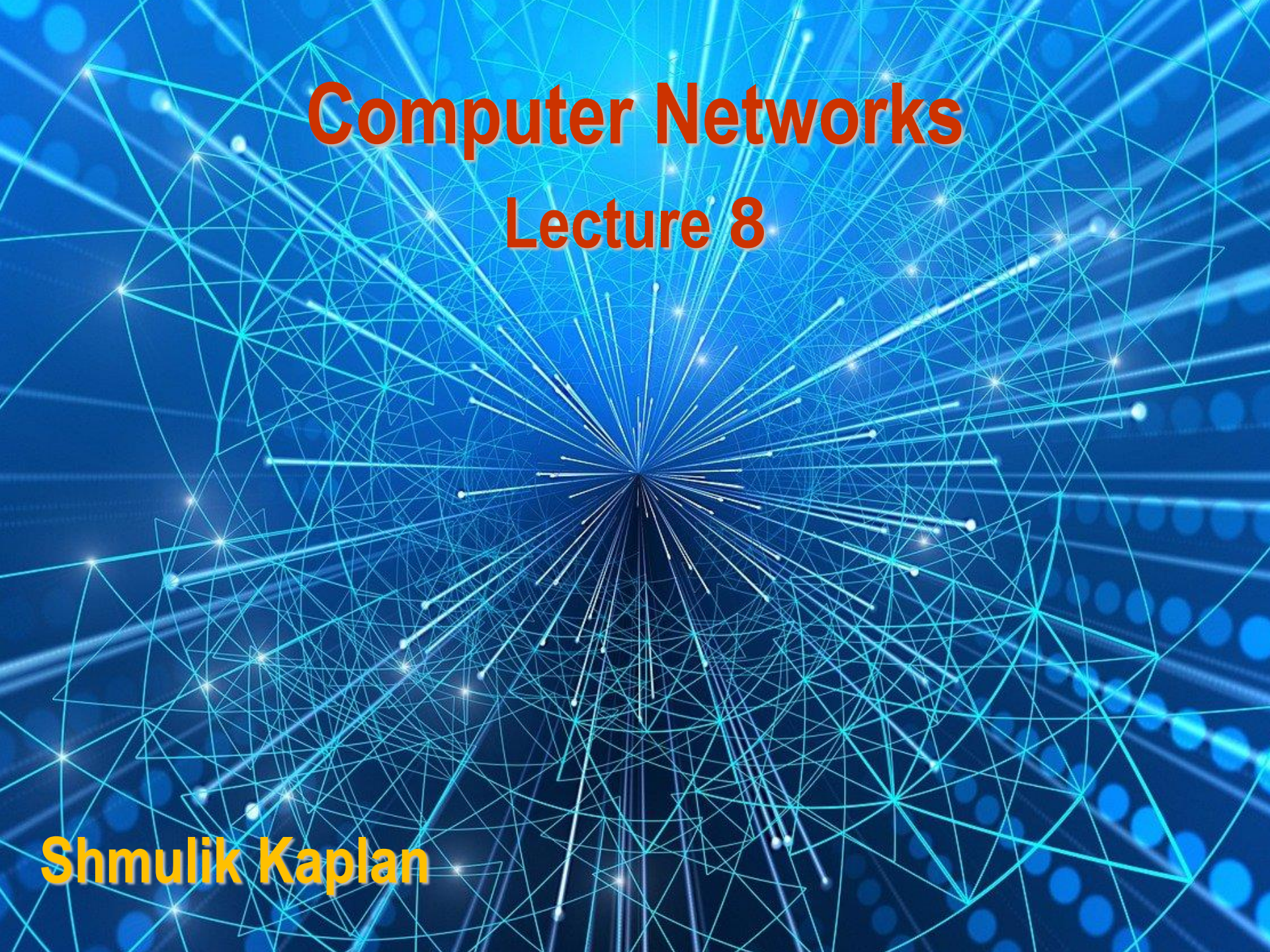

Computer Networks
Lecture 8
Shmulik Kaplan

Today

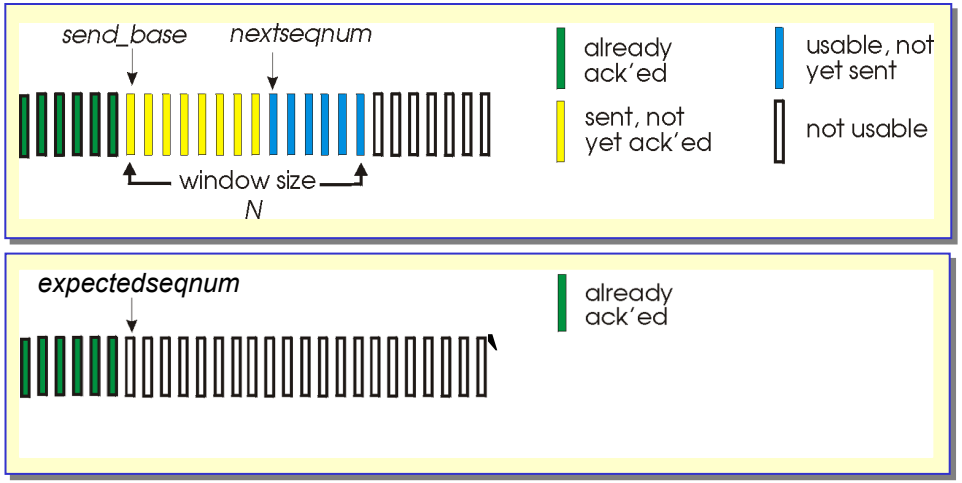
- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - Reorder
  - TCP – The Real thing
  - TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)

Go Back-N - Receiver

- Only **one** packet is acceptable at a time (the next expected).
- The receiver **never stores** out-of-order packets.
- Every ACK is cumulative.

- **When a packet arrives**
- Let the packet's sequence number be seqnum.
- If seqnum == expectedseqnum:
  - Accept the packet.
  - Deliver the data to the application.
  - Increment expectedseqnum.
  - Send an ACK for seqnum (the newly received in-order packet).
- Else (packet is out of order):
  - **Discard** the packet.
    Go-Back-N does not buffer out-of-order segments.
  - Re-send an ACK for the **last correctly received in-order packet**,
    which is expectedseqnum - 1.

- **Sending ACKs**
- ACKs are **cumulative**.
  Each ACK acknowledges *all* packets with sequence
  numbers < acknum.
- Therefore, the ACK number is always expectedseqnum - 1.



Go Back-N - Sender

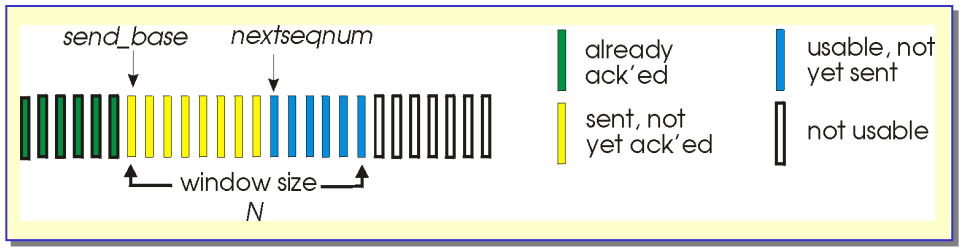
- Only **one** packet is acceptable at a time (the next expected).
- The receiver **never stores** out-of-order packets.
- Every ACK is cumulative.

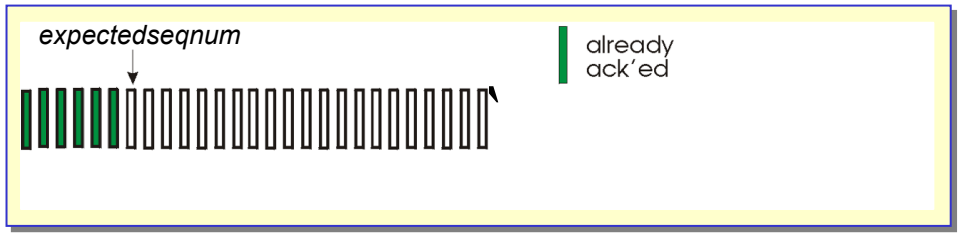
- **If (Exists unsent packs in the win):**
  - Construct packet with sequence number nextseqnum.
  - Send the packet.
  - If base == nextseqnum (window was previously empty), start the timer.
  - Increment nextseqnum.

  Otherwise:
  - The window is full; the sender must wait.

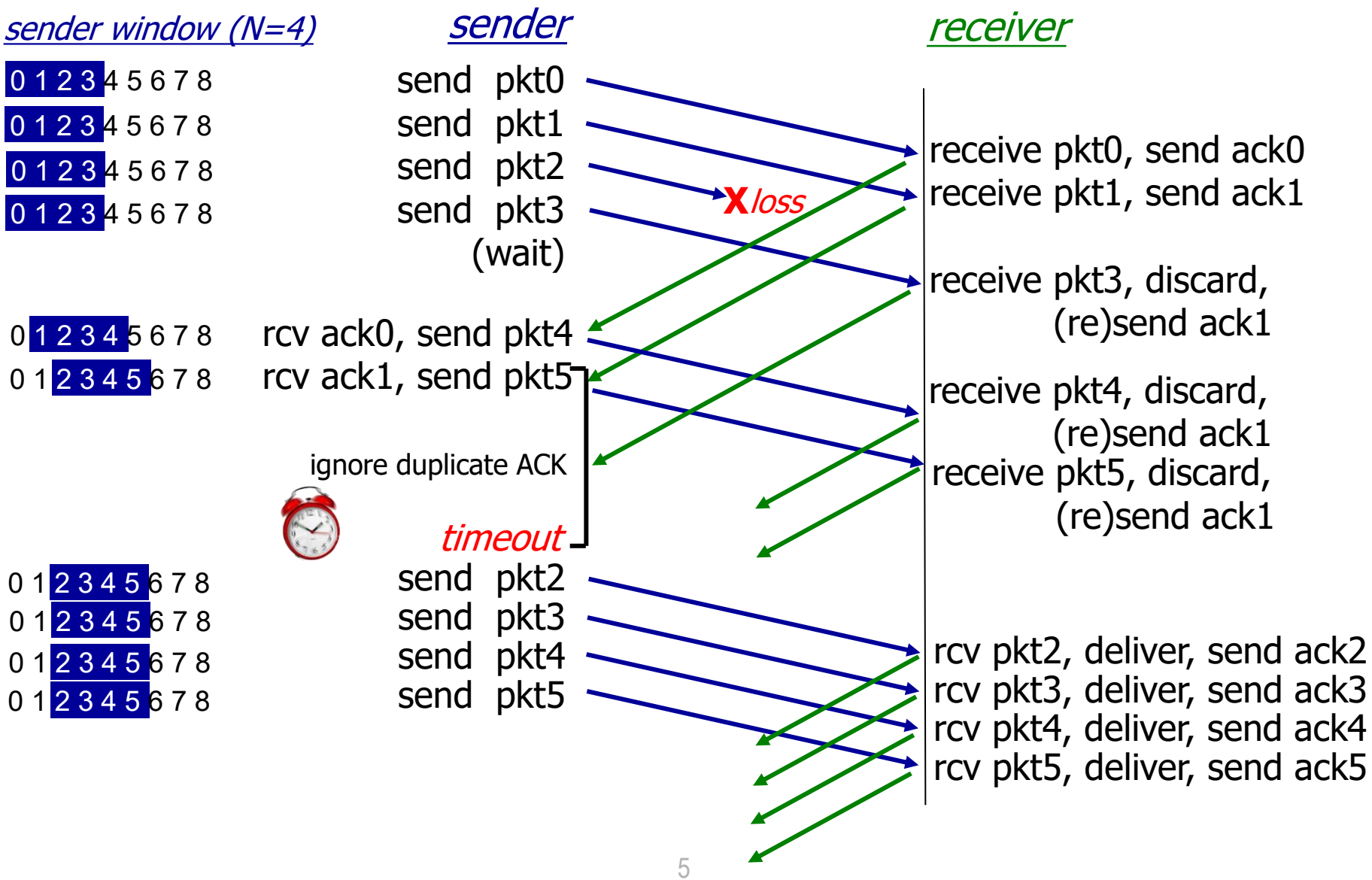
- **When an ACK is received**
- Let acknum be the acknowledged sequence number.
- If acknum >= base:
  - Set base = acknum + 1 (cumulative ACK).
  - If base == nextseqnum:
    stop the timer (no outstanding packets).
  - Otherwise:
    restart the timer for the oldest unacked packet (i.e., sequence base).

- **On Timer Expiration (Timeout Event)**
- The sender **retransmits all packets** in the window:
  - i.e., packets with sequence numbers from base to nextseqnum - 1.
- Restart the timer.





GBN in action



Sliding Window - Selective Repeat

- receiver *individually* acknowledges all correctly received pkts
  - buffers packets, as needed, for eventual in-order delivery to upper layer

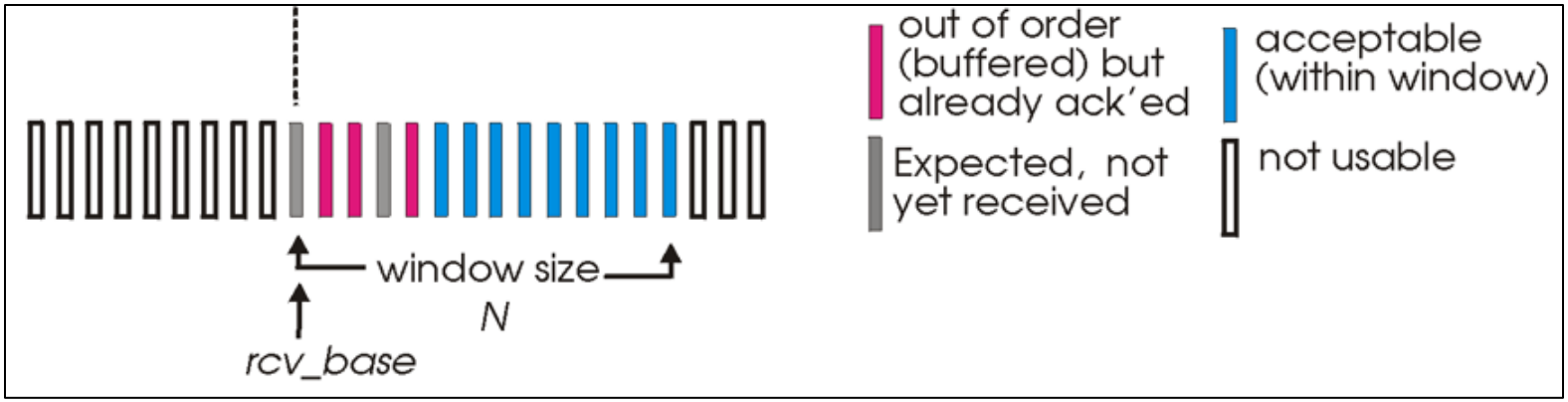
- sender only resends pkts for which ACK not received
  - sender timer for each unACKed packet
- sender window
  - N consecutive seq #'s
  - limits seq #s of sent, unACKed packets

Sliding window – Selective Repeat (Receiver)

- receiver **accepts and buffers out-of-order packets**.
  It delivers data to the application **only when all earlier packets have been delivered**.
- Uses per packet ACK

- **Receiver State**
  - **expectedseqnum** (or **rcv_base**): The lowest sequence number not yet delivered to the application.
  - **N**: window size.
  - **buffer[]**: storage for received but not-yet-delivered packets.
  - **received[]**: boolean flags indicating which packets in the window have arrived.

- **Packet arrives - Case A:**
  seqnum is inside the receiver window
- If the packet was **not previously received**:
  - Store it in the buffer.
  - Mark it as received.
- Send an ACK for seqnum
- If seqnum == rcv_base:
  - deliver full seq of received packets to app.
  - For every packet delivered, slide rcv_base forward..

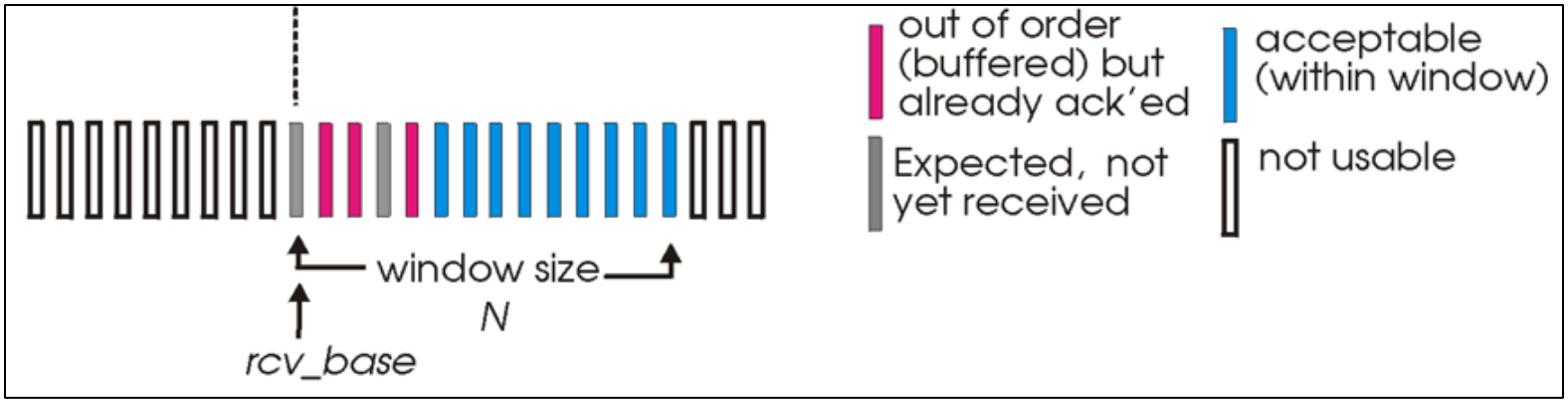


Sliding window – Selective Repeat (Receiver)

- receiver **accepts and buffers out-of-order packets**.
  It delivers data to the application **only when all earlier packets have been delivered**.
- Uses per packet ACK

- **Packet arrives - Case B:**
  seqnum is before the window
  - seqnum < rcv_base
    - This is a **duplicate** of a packet that has already been delivered.
    - Sender might need the ACK again.
  - Send an ACK for seqnum.

- **Packet arrives - Case C:**
  seqnum is above the window
  - seqnum ≥ rcv_base + N
    - Packet is outside the window and cannot be accepted.
    - Discard it.



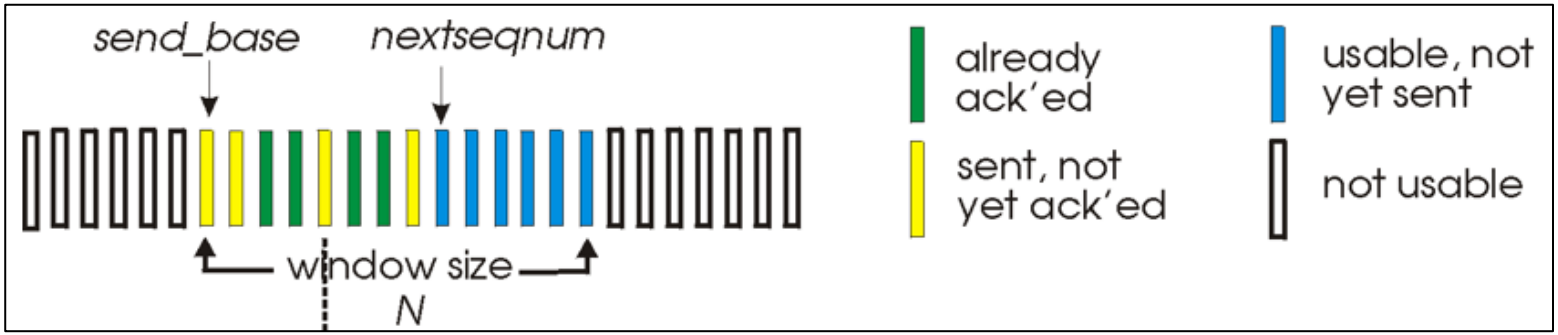
Sliding window – Selective Repeat (Sender)

- sender keeps a separate timer **per outstanding packet** and may retransmit each one individually. Its window allows transmission of up to **N unacknowledged packets** at a time.

- **Sender State**
  - **base** (or **snd_base**): The lowest sequence number of any unacknowledged packet.
  - **nextseqnum**: The next sequence number for sending.
  - **N**: Window size.
  - **packet[]**: A buffer containing sent-but-not-yet-acknowledged packets.
  - **timer[]**: Individual timers for each outstanding packet.

- **If the window is not full:**
  - Create a packet with sequence number nextseqnum.
  - Store it in the sender's buffer.
  - Send the packet.
  - Start a **per-packet timer** for nextseqnum.
  - Increment nextseqnum.

  Otherwise (window full):
  - The sender must wait; no transmission permitted until an ACK advances the window.



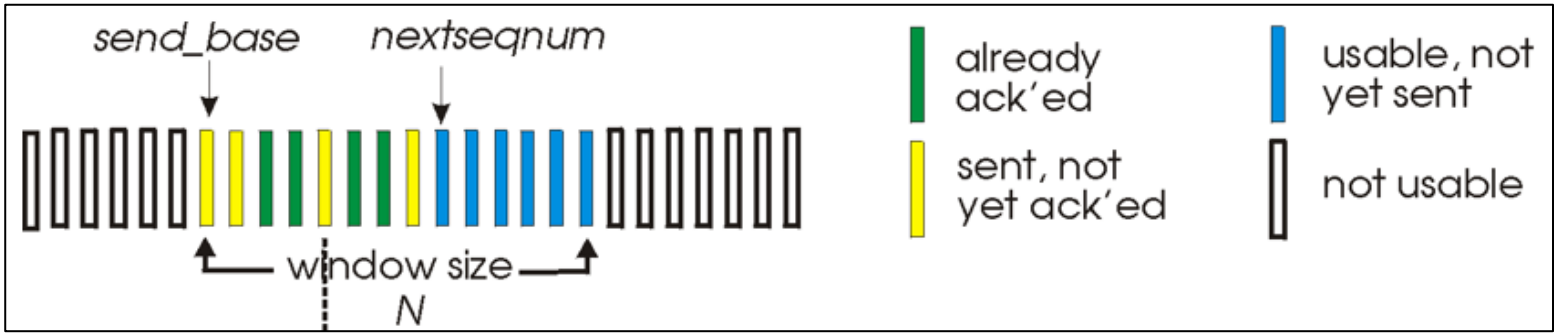
Sliding window – Selective Repeat (Sender)

- sender keeps a separate timer **per outstanding packet** and may retransmit each one individually. Its window allows transmission of up to **N unacknowledged packets** at a time.

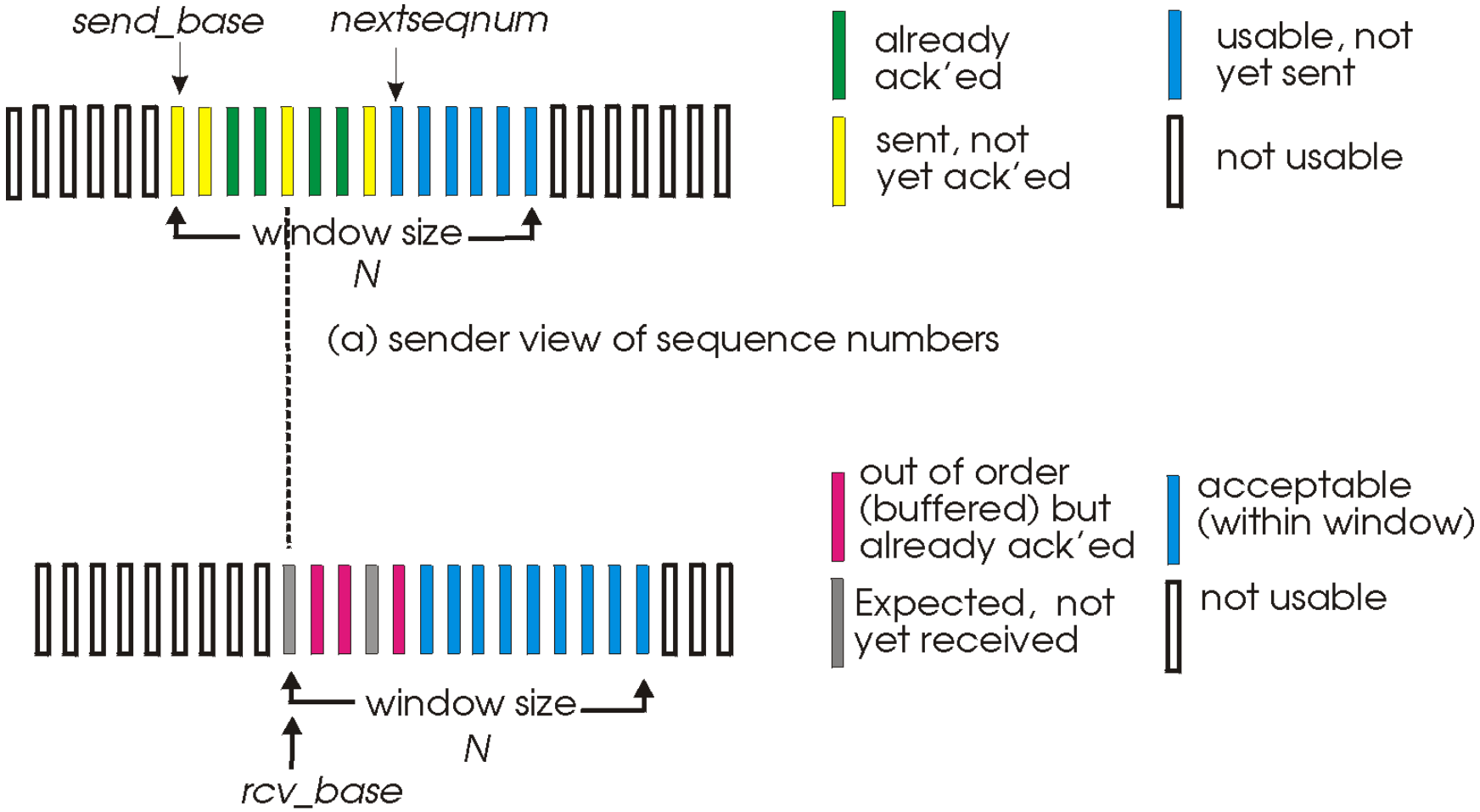
- If acknum is within the sender window:
  - Mark packet acknum as acknowledged.
  - Stop the timer for acknum.
  - If acknum == snd_base:
  - Move snd_base to the smallest sequence number not yet ACKed.
- ACKs outside the sender window are ignored.

- **On Timer Expiration (per packet)**

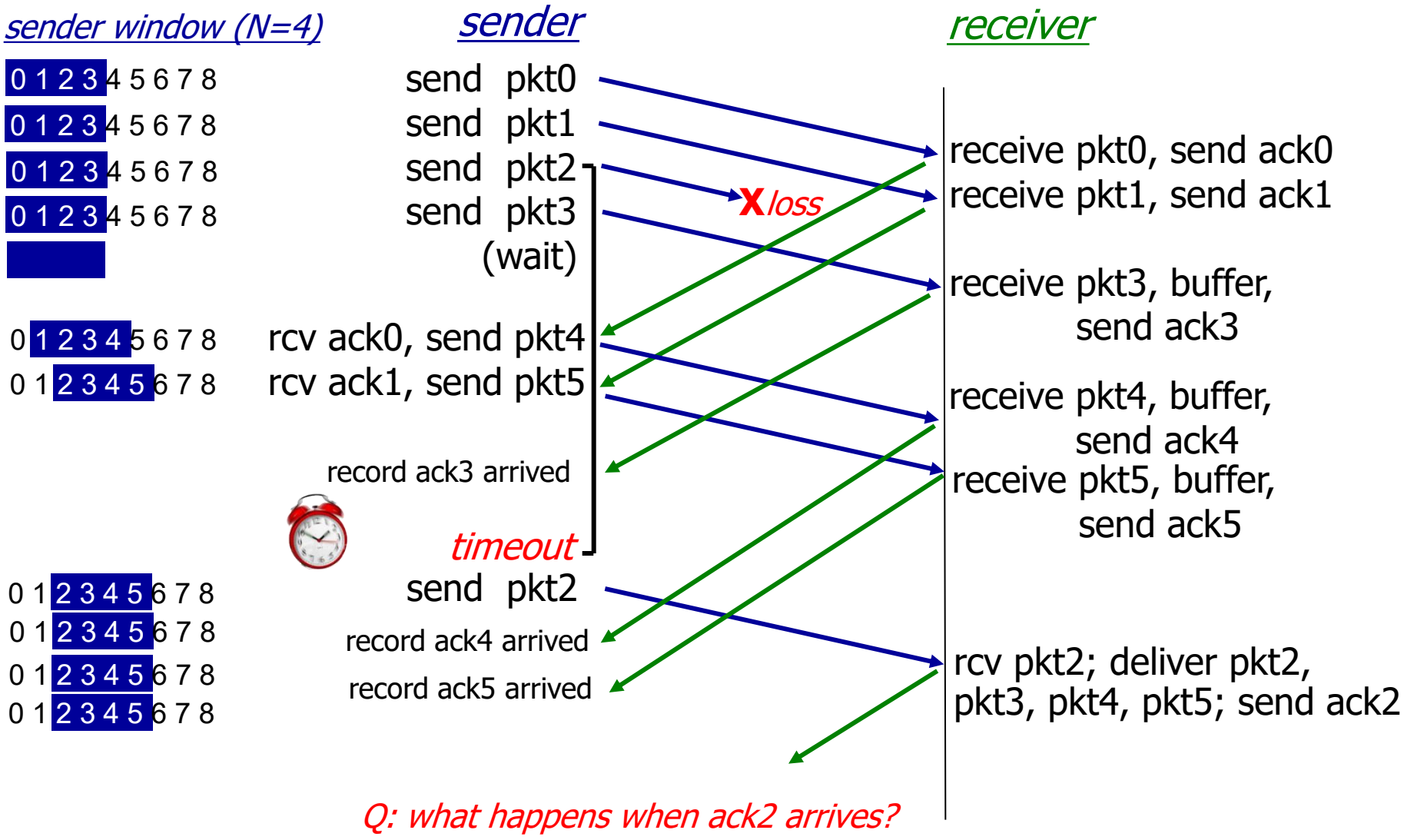
  For a timeout of packet i:
  - Retransmit packet i.
  - Restart timer for packet i.
- No other packets are retransmitted. This is the core difference from Go-Back-N.



Selective repeat: Sender, Receiver Windows



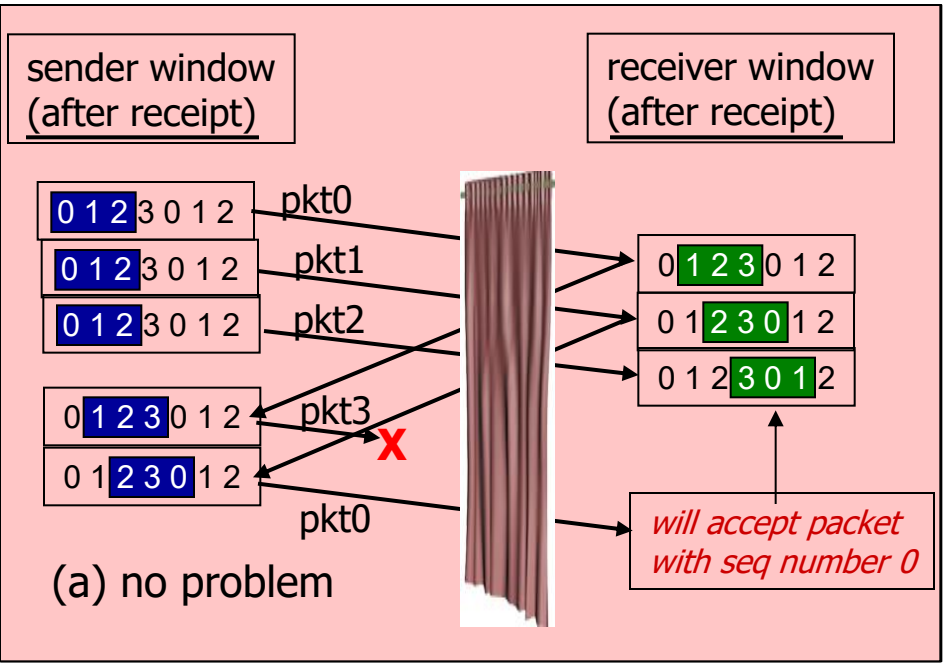
Selective Repeat Example

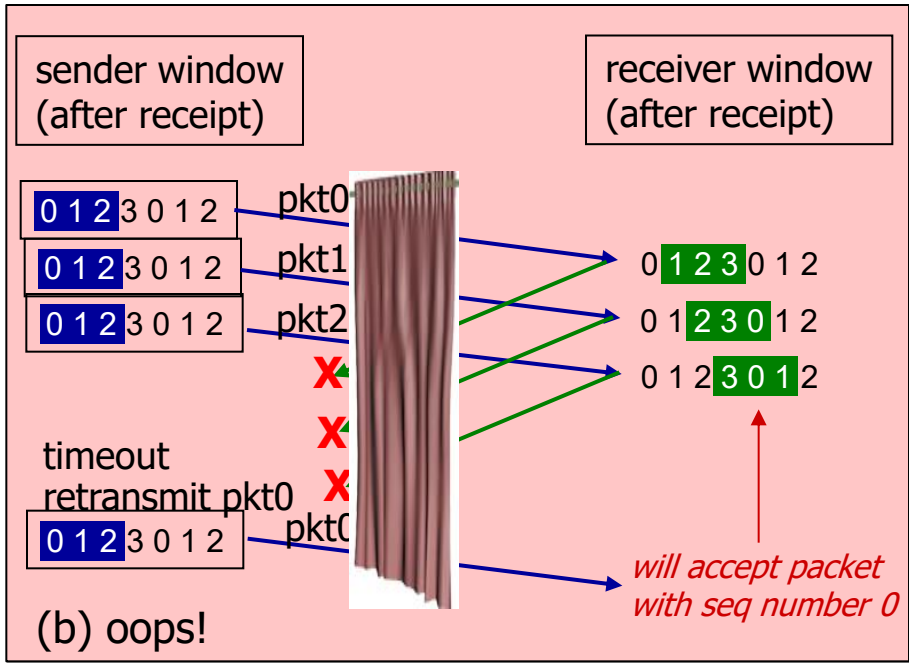


Selective Repeat – How many seq nums are needed?

example:

- seq #'s: 0, 1, 2, 3
- window size=3





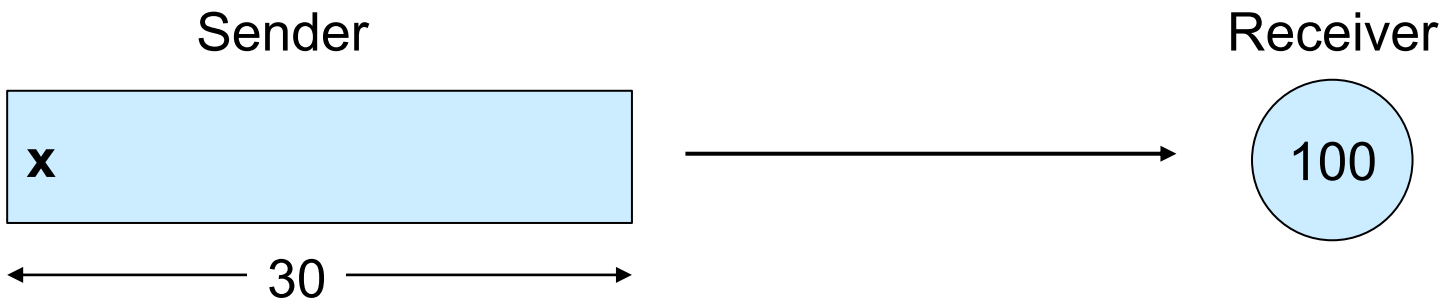
receiver can't see sender side.
receiver behavior identical in both cases!
something's (very) wrong!

A Question From Final Exam

השאלה מתייחסת לאלגוריתם Go-Back-N , כאשר גודל החלון של השולח הוא **30** ותחום ערכי המספרים הסידורים (sequence numbers) הוא מ-1 עד 2,048. הנח שבזמן מסוים שנסמן אותו ב- *t* המקבל מחכה לקבל הודעה עם מספר סידורי **100** (ז״א המקבל כבר קיבל את כל ההודעות עד מספר סידורי 99 כולל).

א. מהם כל הערכים הסידורים האפשריים שיכולים להופיע <u>במקום הראשון</u> בחלון של השולח בזמן *t* ? נמק את תשובתך.

ב. מהם כל הערכים הסידוריים האפשריים בהודעות ה- ACK הנמצאות בתנועה מהמקבל אל השולח בזמן *t* ? נמק את תשובתך. הנח שהודעות ה- ACK מגיעות לשולח <u>בסדר בו נשלחו</u> על ידי המקבל.

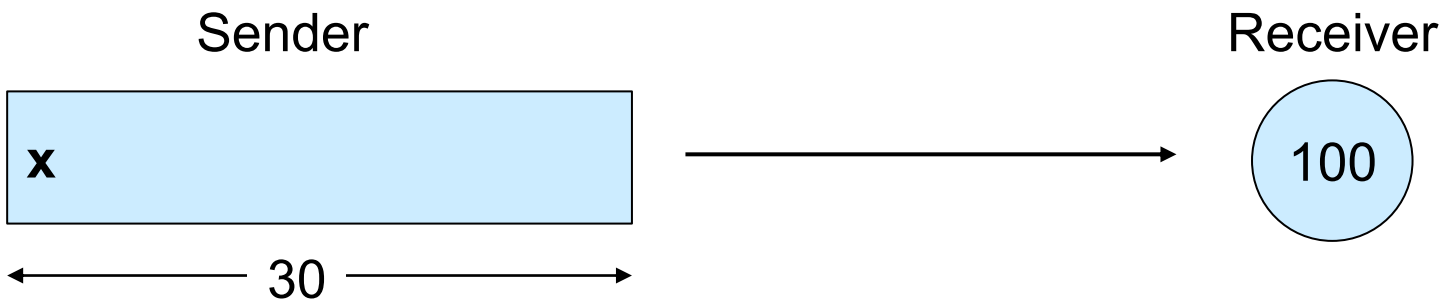


A Question From Final Exam

Consider the Go_Back-N protocol with a sender widow size of **30** and sequence number range from 1 to 2,048. Suppose that a time ***t***, the next in-order packet that the receiver is expecting has a sequence number **100**. (That is the receiver has already accepted all the messages up to 99.)

1. What are the possible sequence numbers that can appear at the beginning of the sender's window at time *t* ? Justify your answer.

2. What are all the possible values of the ACK field in all possible messages currently propagating back from the receiver to the sender at time *t* ? Justify your answer. (Assume that the medium does not reorder messages.)



A Question From Final Exam

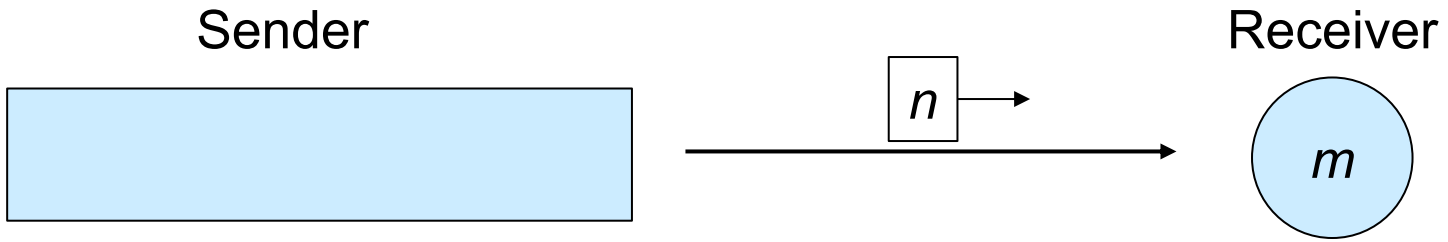
הנח שאנחנו משנים את אלגוריתם **Go-Back-N** של **המקבל** באופן הבא:
נניח שהמקבל מחכה לקבל חבילה עם מספר סידורי ***m***. (ז"א, המקבל כבר קיבל את כל החבילות עם מספר סידורי הקטן מ- m באופן תקין ולפי הסדר). בשלב זה, כאשר מגיעה חבילה תקינה עם מספר סידורי ***n*** , אם $n=m$ המקבל מתנהג כמו באלגוריתם המקורי ושולח ACK עבור חבילה n . אחרת:

עבור כל אחד משלושת המקרים, החליטו כיצד השינוי משפיע על נכונות האלגוריתם. נמק את תשובתך.

מקרה א: אם $n > m$, המקבל זורק את החבילה **וגם** שולח שוב ACK עבור חבילה m-1 ;
אם $n < m$ המקבל זורק את החבילה ולא שולח ACK.

מקרה ב: אם $n > m$, המקבל זורק את החבילה ולא שולח ACK ;
אם $n < m$ המקבל זורק את החבילה **וגם** שולח שוב ACK עבור חבילה m-1 .

מקרה ג: אם $n > m$, המקבל זורק את החבילה ולא שולח ACK ;
אם $n < m$ המקבל זורק את החבילה ולא שולח ACK.



A Question From Final Exam

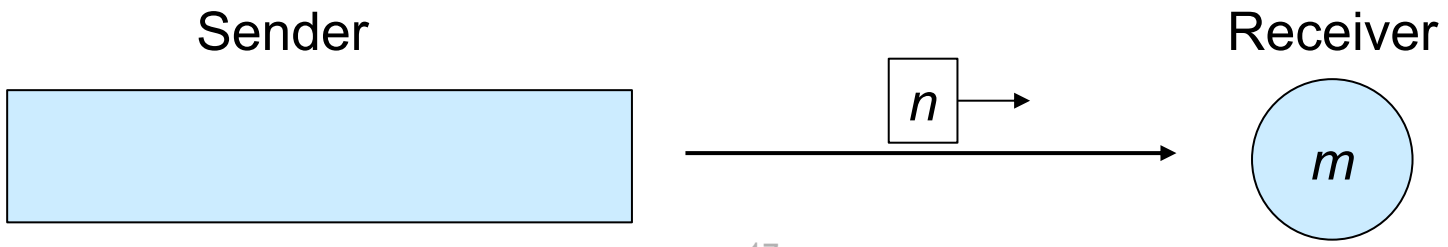
Assume that we change the **Go-Back-N** algorithm of the **receiver**, as follows: Let's assume that the receiver is now waiting to receive packet with sequence number ***m***. (Than is, the receiver has already received correctly and in-order all the packets with sequence numbers smaller than *m*.) At that point, when a packet with sequence number ***n*** is received correctly then, if $n=m$ the receiver behave as in the original algorithm and sends an ACK for packet *n*. Otherwise:

For each one of the three cases decided how the change affects the correctness of the algorithm. Justify your answer.

1. if $n > m$, the receiver discards the packet **and** resends an ACK for packet m-1; if $n < m$, the receiver discards the packet **and** does not send an ACK.

2. if $n > m$, the receiver discards the packet **and** does not send an ACK; if $n < m$, the receiver discards the packet **and** resends an ACK for packet m-1.

3. if $n > m$, the receiver discards the packet **and** does not send an ACK; if $n < m$, the receiver discards the packet **and** does not send an ACK.



Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - ✓ Reorder
  - TCP – The Real thing
  - TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)

Go-Back-N ♦ Selective Repeat ♦ TCP

- ACKs are cumulative ?
  - Go-back-N: Yes
  - Selective Repeat: No
  - TCP: Yes

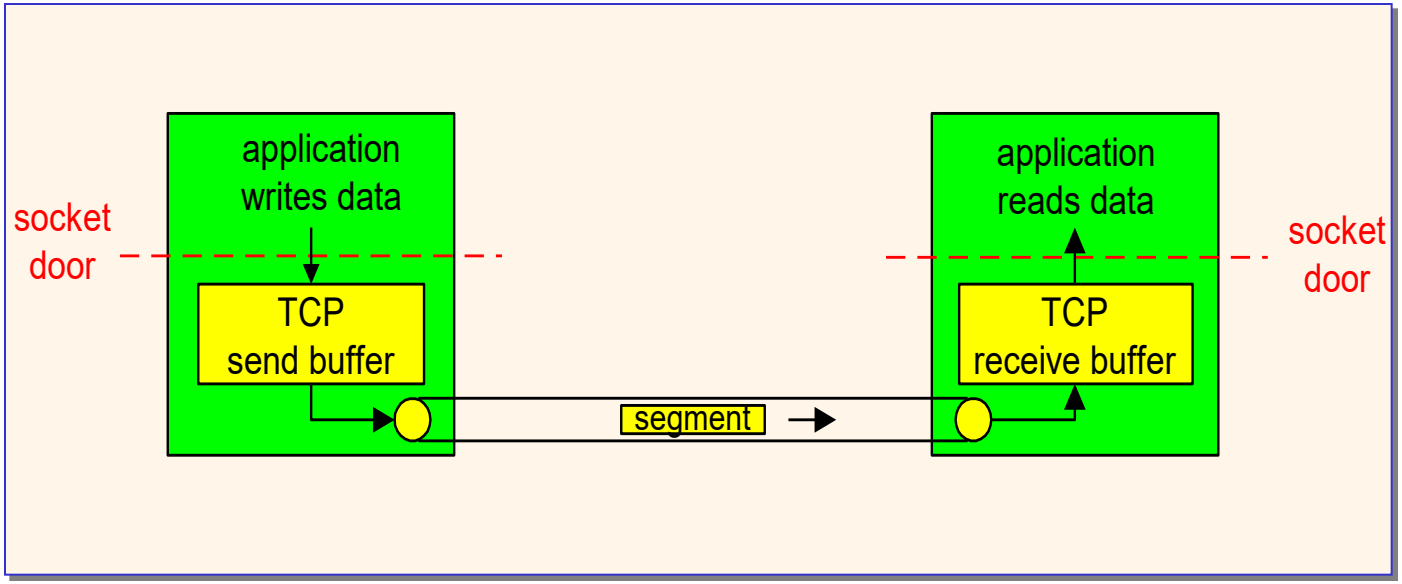
- When one segment is lost, retransmit one segment ?
  - Go-back-N: No
  - Selective Repeat: Yes
  - TCP: Yes

- Must buffer, received but out of order segments ?
  - Go-back-N: No
  - Selective Repeat: Yes
  - TCP: No, but

- One timer ?
  - Go-back-N: Yes
  - Selective Repeat: No
  - TCP: Yes

TCP: Overview RFCs: 793, 1122, 1323, 2018, 2581

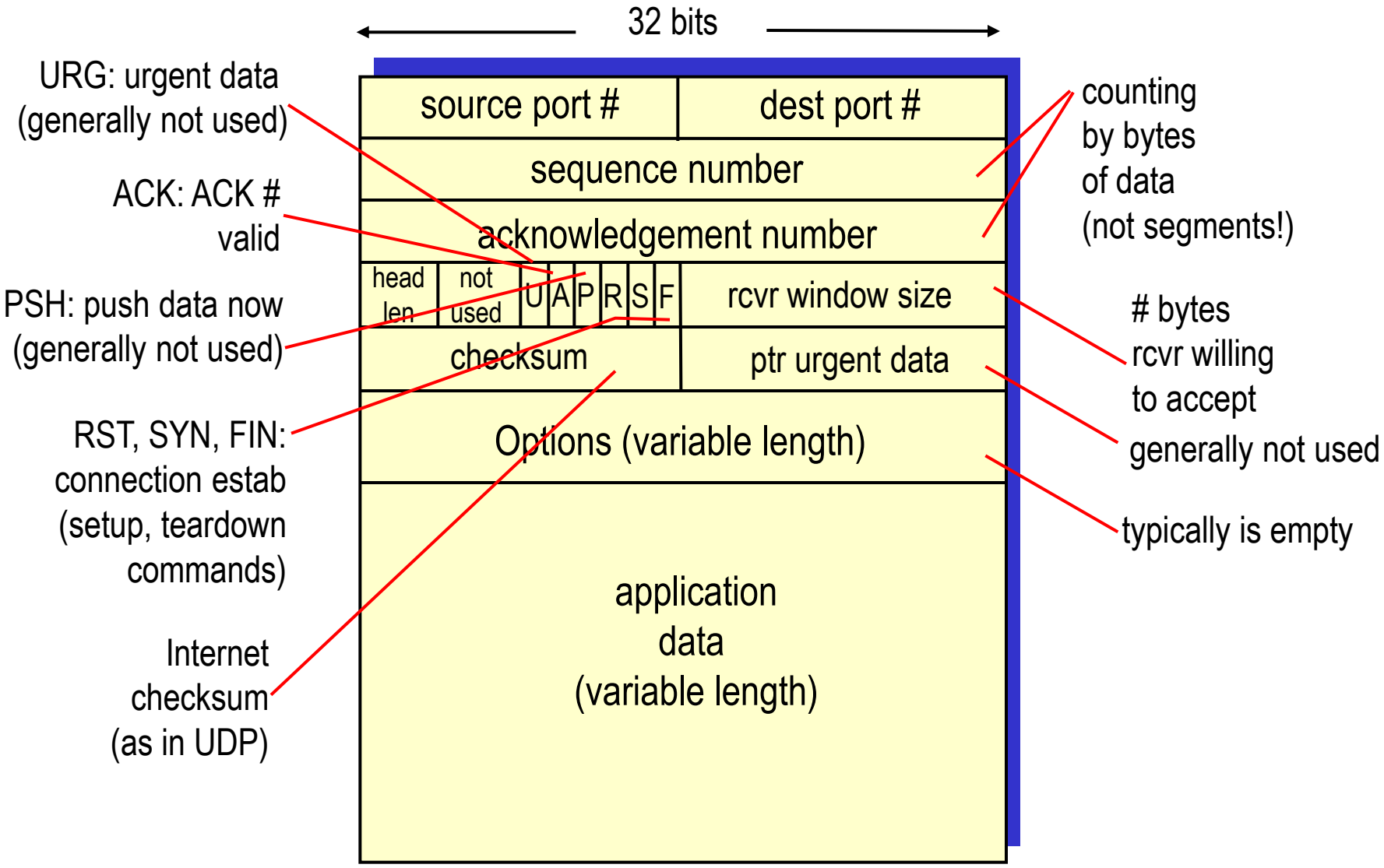
- point-to-point -- one sender, one receiver
- reliable, in-order *byte steam*
- TCP congestion and flow control set window size
- *send & receive buffers*



TCP: Overview (continued)

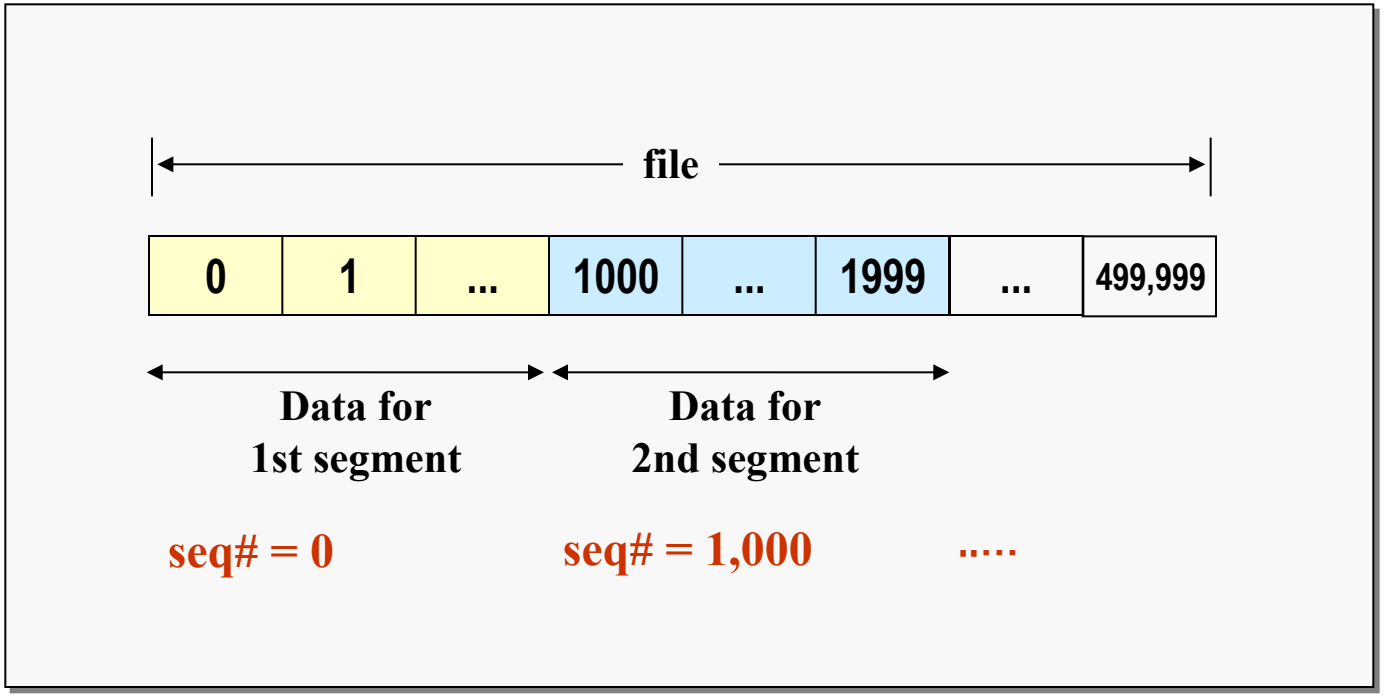
- Full duplex data
  - bi-directional data flow in same connection
- MSS: maximum segment size
  - the largest amount of data, specified in bytes (not including TCP headers)
  - implementation dependent (determined by OS)
  - common values are 1500 bytes, 536 bytes, 512 bytes (data only)
- Connection-oriented --- three-way handshake
  - handshaking (exchange of control ,messages) between sender and receiver before data exchange
- Flow controlled
  - sender will not overwhelm receiver
- TCP congestion control
  - "too many sources sending too much data too fast for *network* to handle"

TCP segment structure



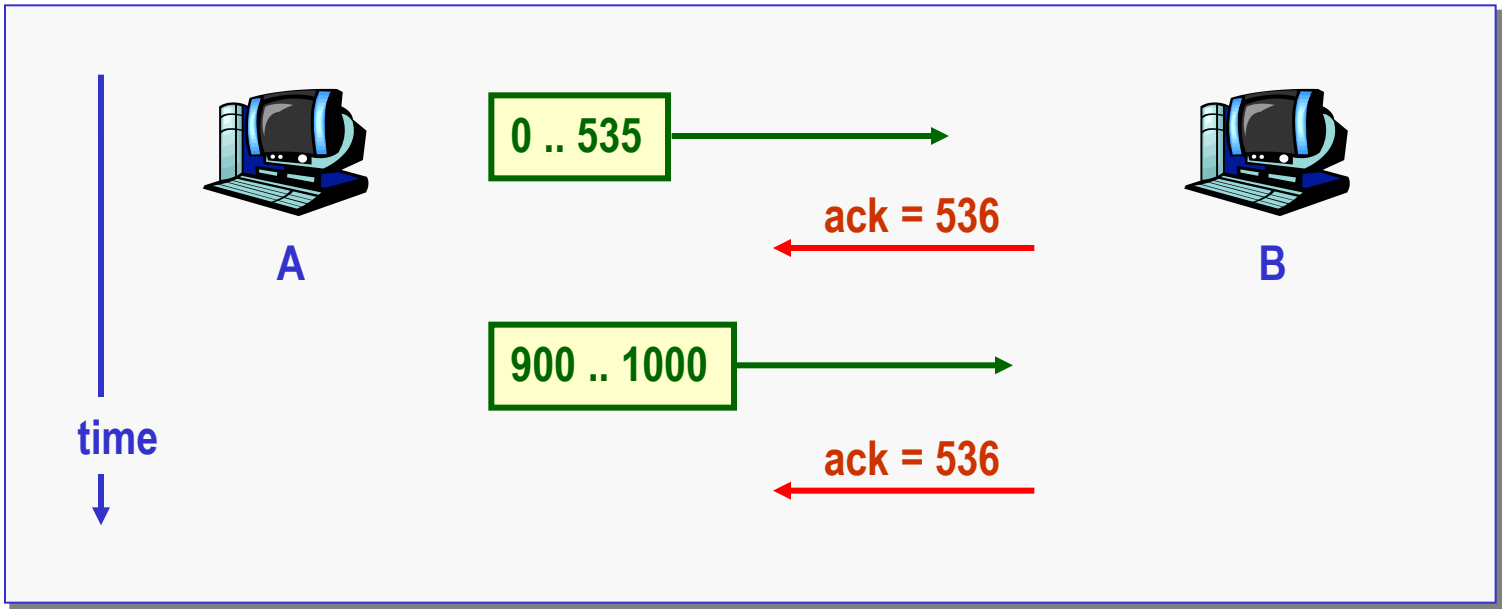
Dividing file data into TCP segments

File = 500,000 bytes
MSS = 1,000 bytes



Acknowledgment number

- The acknowledgment number that B puts in its segment is the seq. number of the next byte B is expecting from A
- Cumulative acknowledgments: TCP ack bytes up to the first missing byte in the stream



Example: telnet

- Each char typed by the user is sent to the remote host
- The remote host sends back a copy of each char
- Only than the char is displayed on the user screen



Simple telnet scenario

(TCP seq. #'s and ACKs)

Seq. #'s:

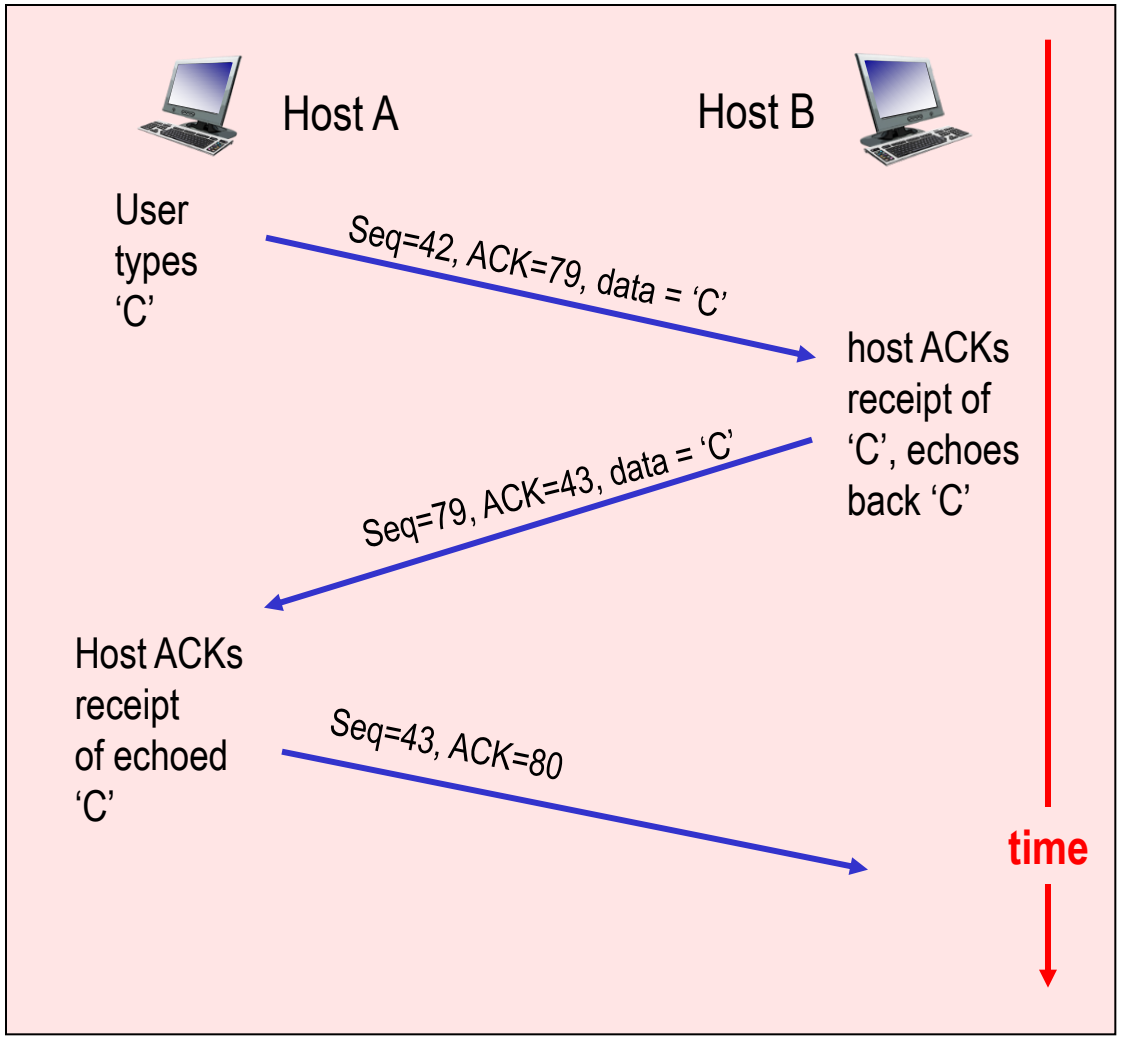
- byte stream "number" of first byte in segment's data

ACKs:

- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

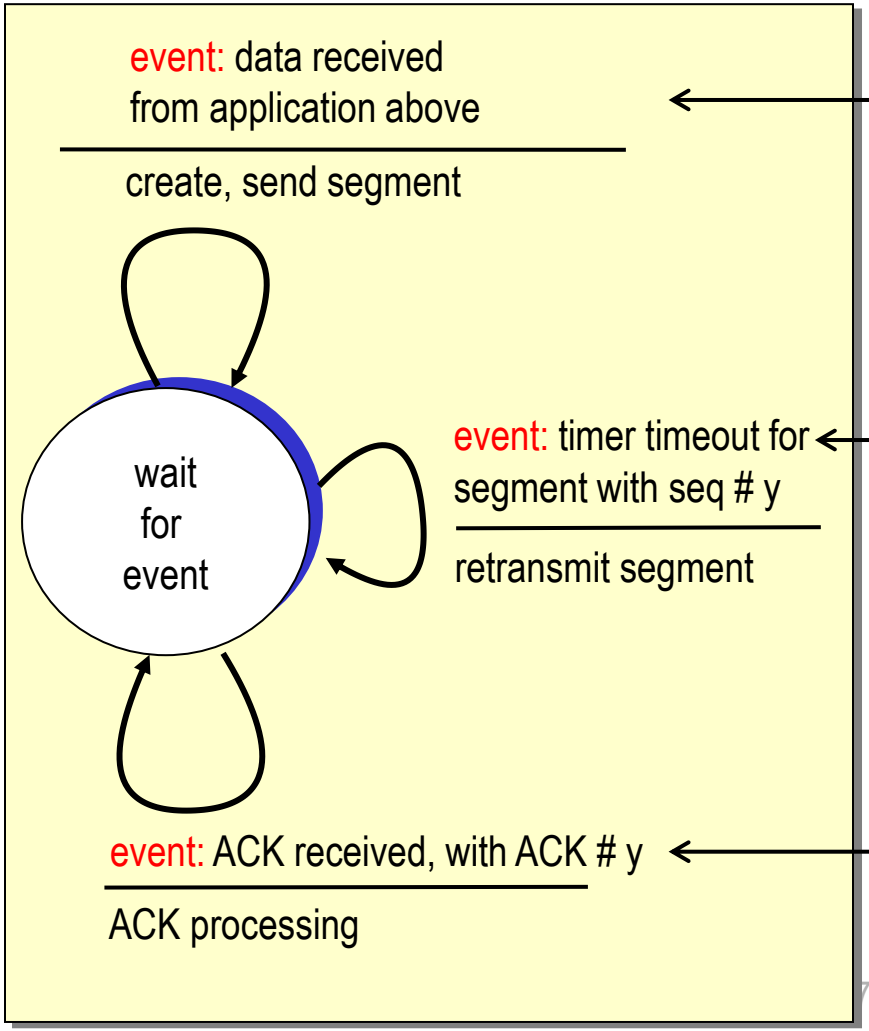
A: TCP spec doesn't say, - up to implementor



TCP sender events

Simplified sender

- one way data transfer
- no flow control
- no congestion control



Data received from application

- Create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running (think of timer as for oldest unacked segment)
- expiration interval: `TimeOutInterval`

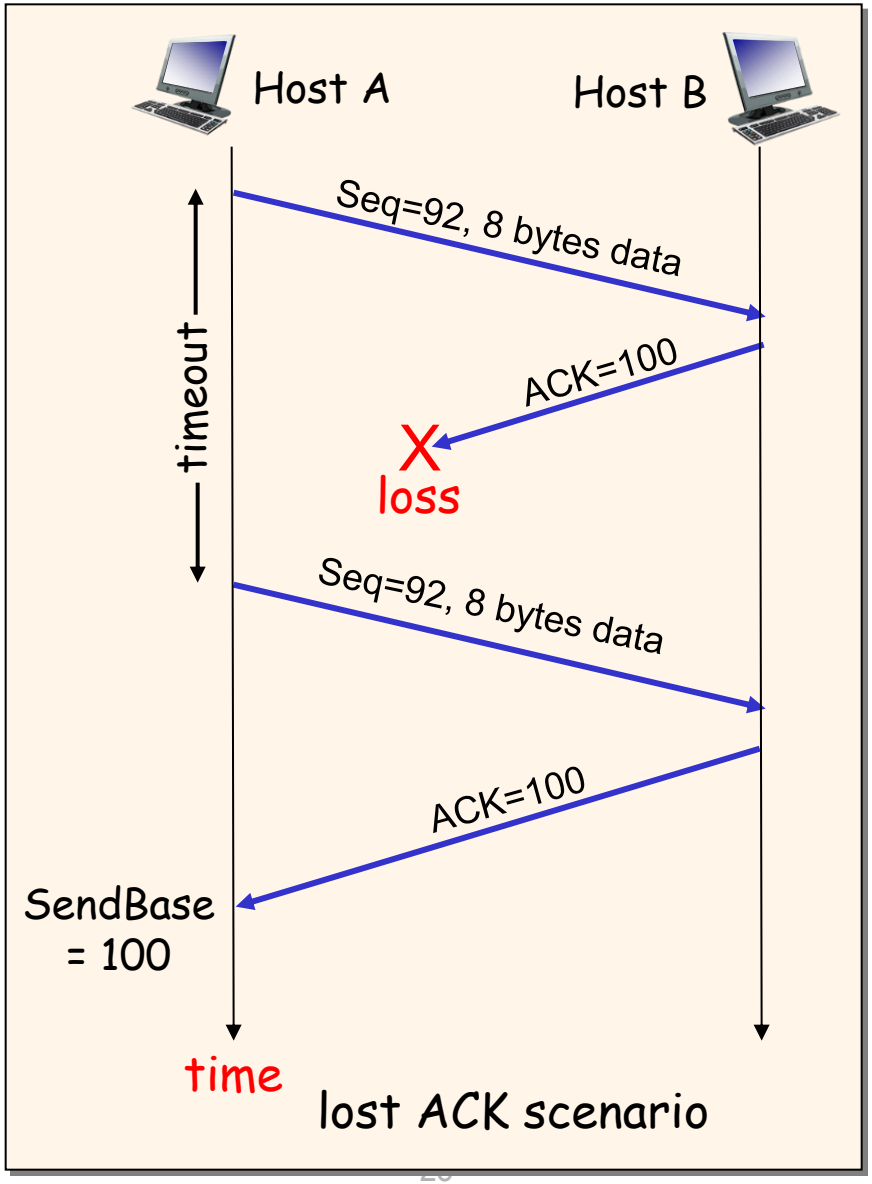
Timeout

- retransmit segment that caused timeout
- restart timer

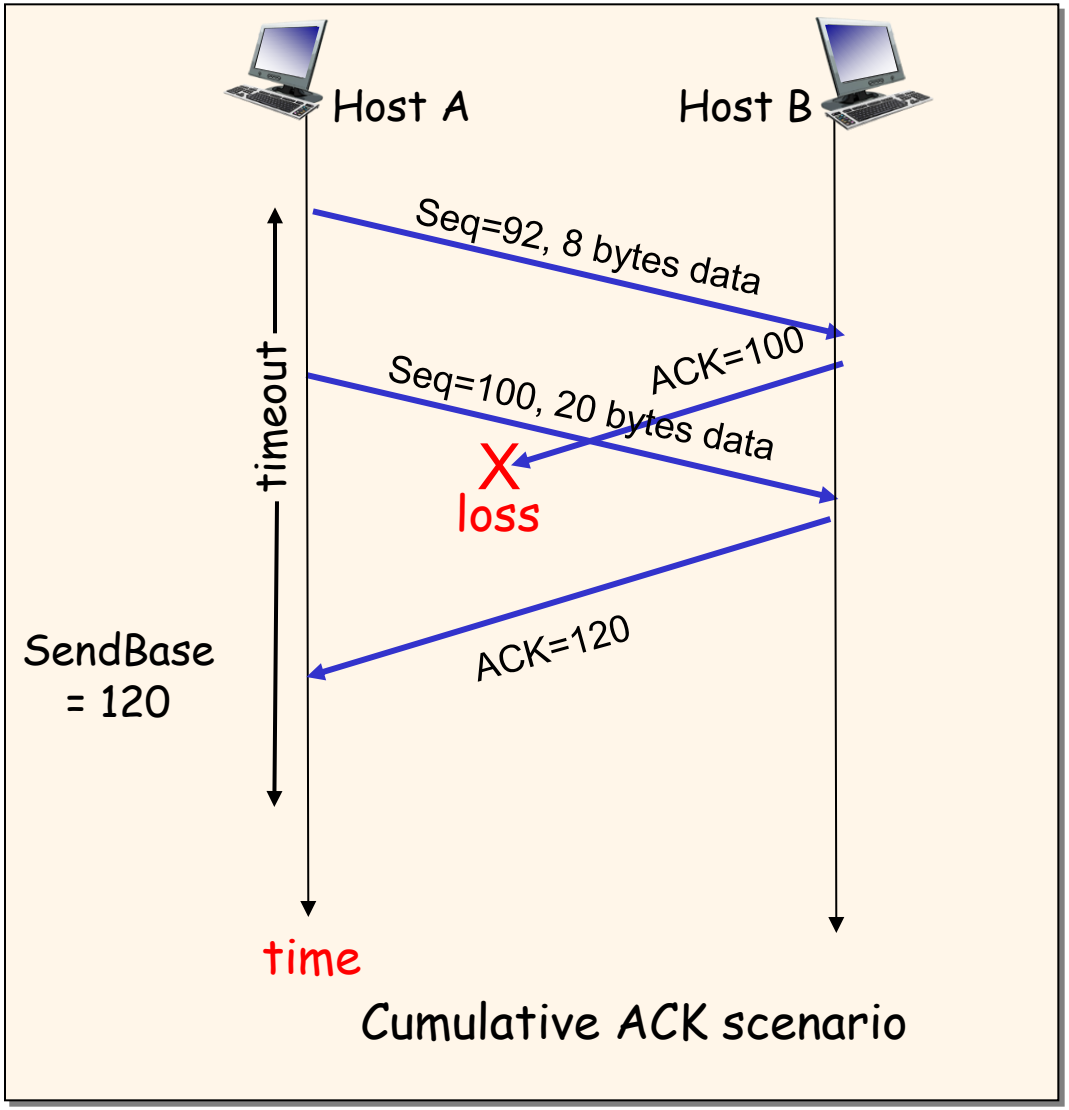
Ack received

- If acknowledges previously unacked segments
  - update what is known to be acked
  - start timer if there are outstanding segments

TCP: retransmission scenarios



TCP retransmission scenarios (more)

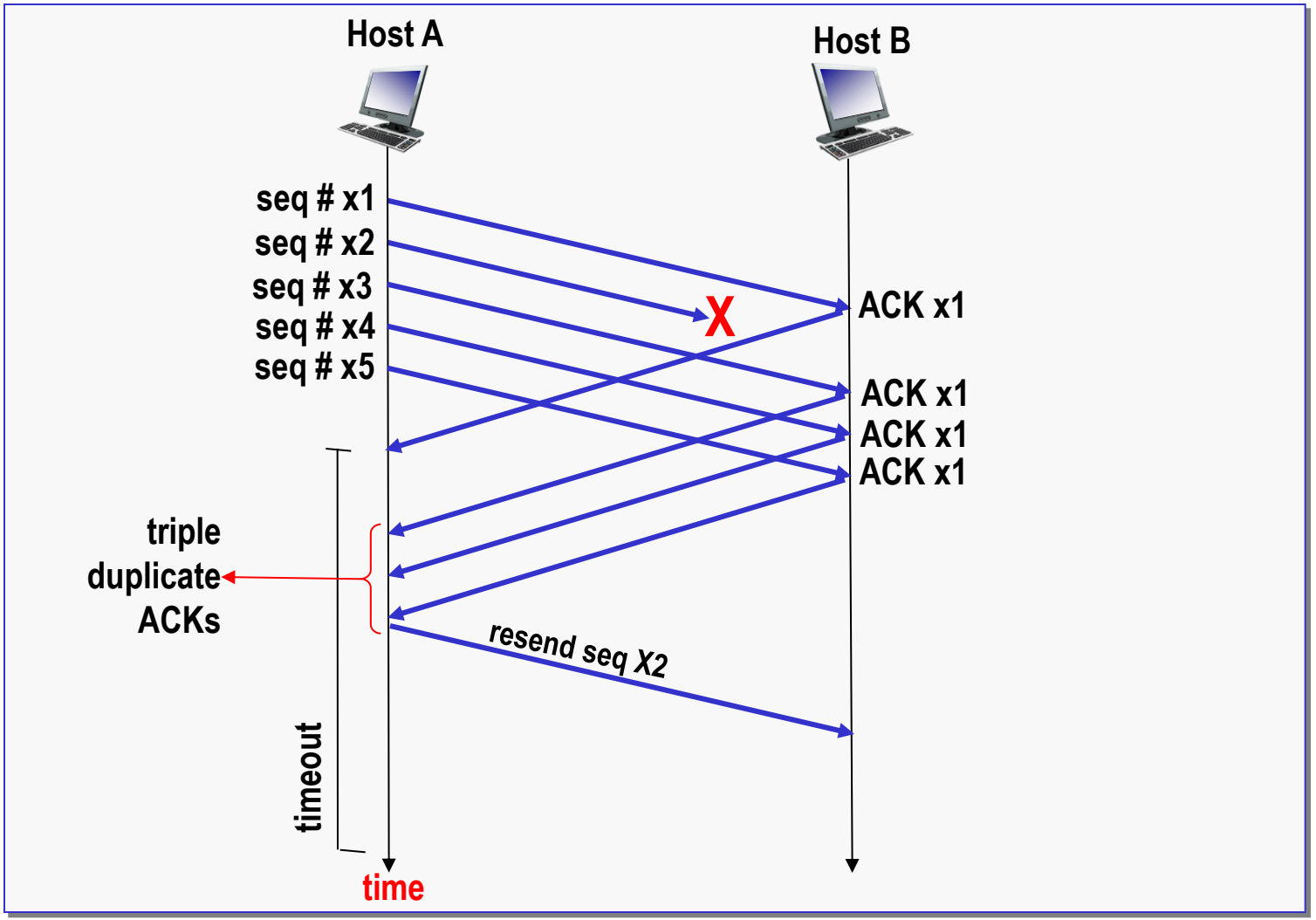


TCP ACK generation [RFC 1122, RFC 2581]

Event at Receiver	TCP Receiver action
Arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	Delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
Arrival of in-order segment with expected seq #. One other segment has ACK pending	Immediately send single cumulative ACK, ACKing both in-order segments
Arrival of out-of-order segment higher-than-expect seq. # . Gap detected	Immediately send duplicate ACK, indicating seq. # of next expected byte
Arrival of segment that partially or completely fills gap	Immediate send ACK, provided that segment starts at lower end of gap

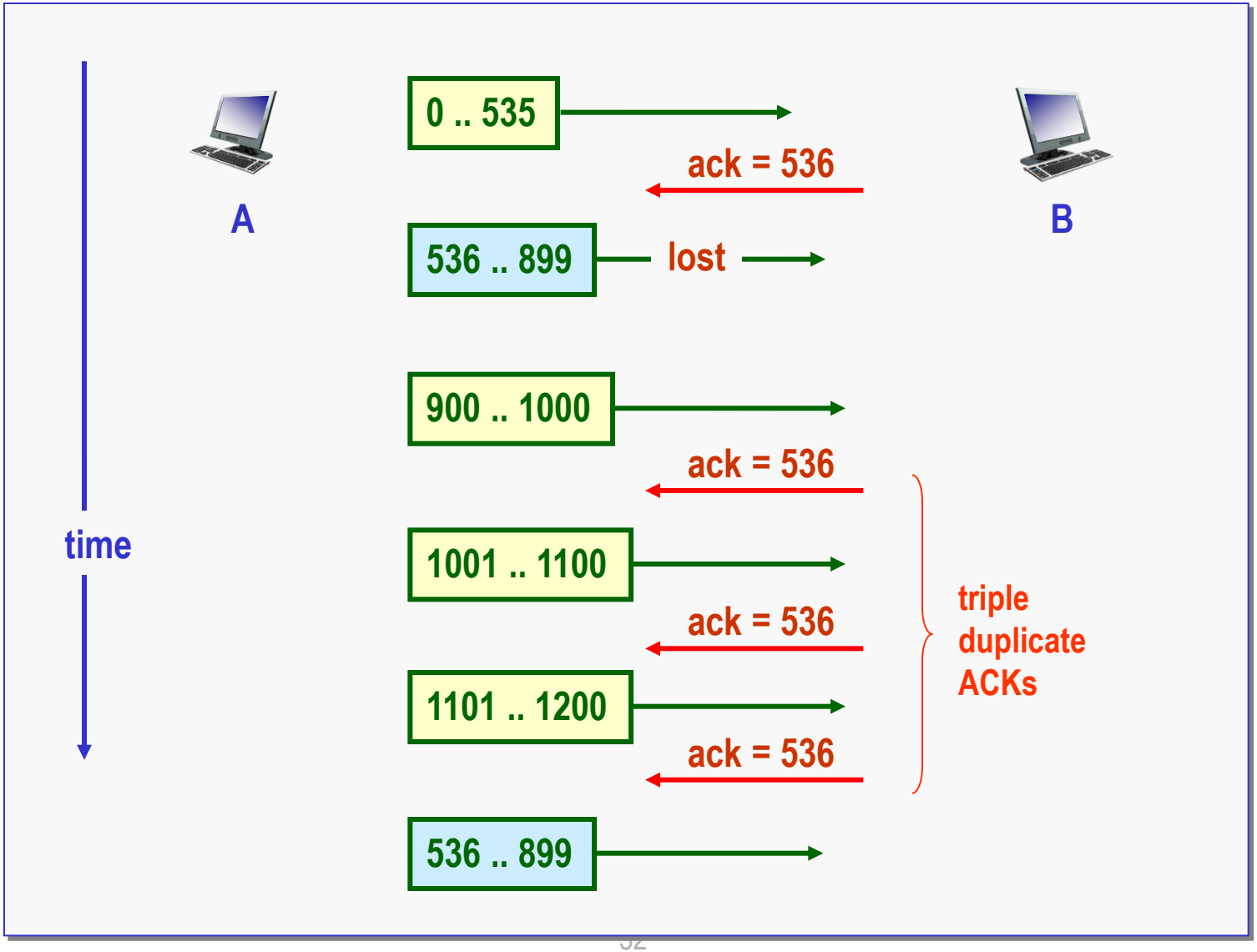
Fast Retransmit

Resend segment before timer expires



Fast Retransmit

Resend segment before timer expires



Segment retransmition

When there is a timeout event, or a triple duplicate ACK event TCP always retransmits one segment

TCP Round Trip Time and Timeout

Q: How to set TCP timeout value?

- longer than RTT
  - note: RTT will vary
- too short: premature timeout
  - unnecessary retransmissions
- too long: slow reaction to segment loss

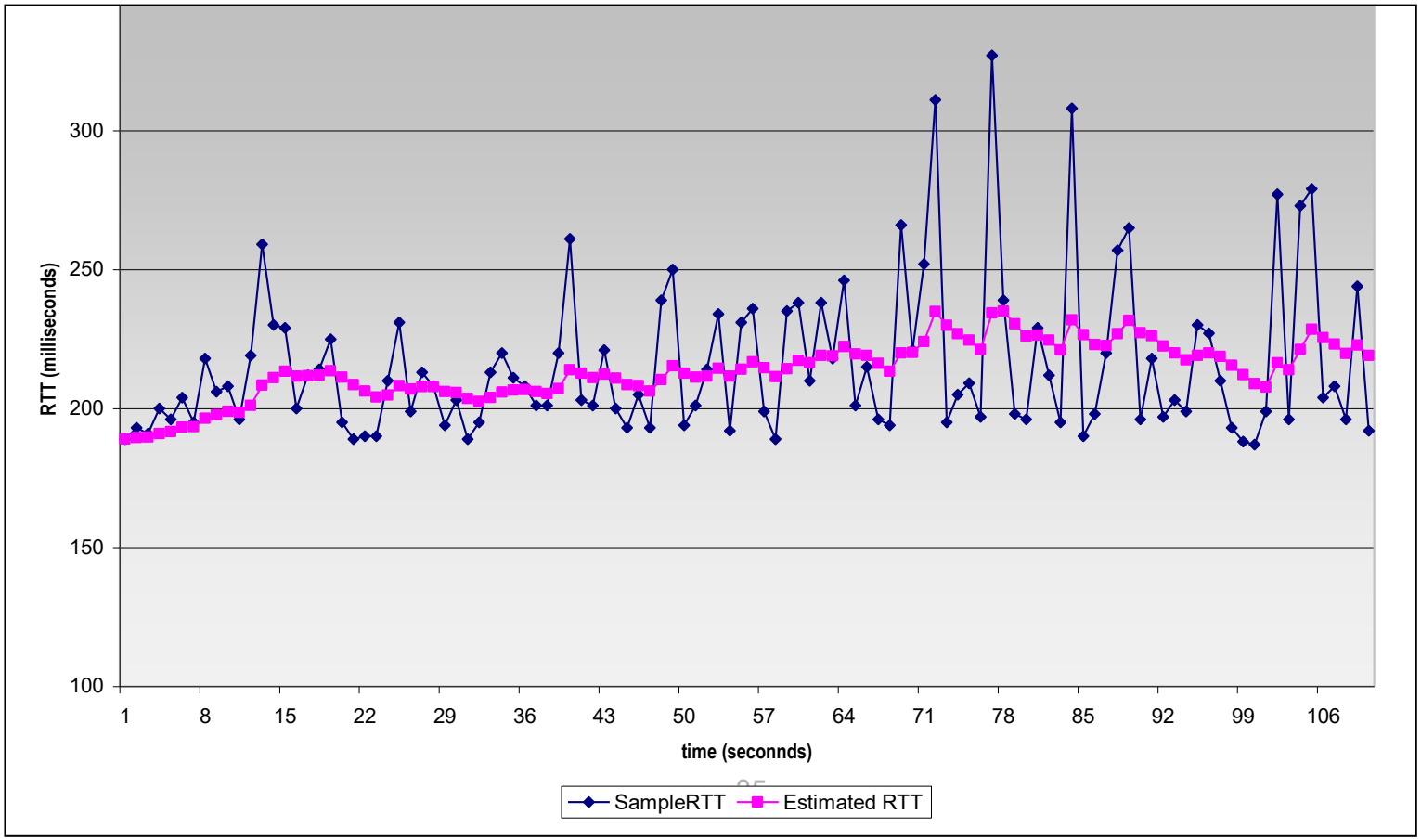
Q: How to estimate RTT?

- **`SampleRTT`**: measured time from segment transmission until ACK receipt
  - ignore retransmissions, cumulatively ACKed segments
- **`SampleRTT`** will vary, want estimated RTT "smoother"
  - use several recent measurements, not just current **`SampleRTT`**

TCP Round Trip Time and Timeout

`EstimatedRTT = (1-x)*EstimatedRTT + x*SampleRTT`

- Exponential weighted moving average
- influence of given sample decreases exponentially fast
- typical value of x: 0.125



TCP Round Trip Time and Timeout

Setting the timeout:

- **EstimtedRTT** plus "safety margin"
  - large variation in **EstimatedRTT** -> larger safety margin
- first estimate of how much SampleRTT deviates from EstimatedRTT:

```
DevRTT = (1-β)*DevRTT +
                β*|SampleRTT-EstimatedRTT|

(typically, β = 0.25)
```

Then set timeout interval:

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```



estimated RTT

"safety margin"

Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - ✓ Reorder
  - ✓ TCP – The Real thing
  - TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)

TCP Connection Management

TCP sender and receiver establish "connection" before exchanging data segments

- initialize TCP variables:
  - seq. #s
  - buffers, flow control info (e.g. **`RcvWindow`**)
- *client:* connection initiator

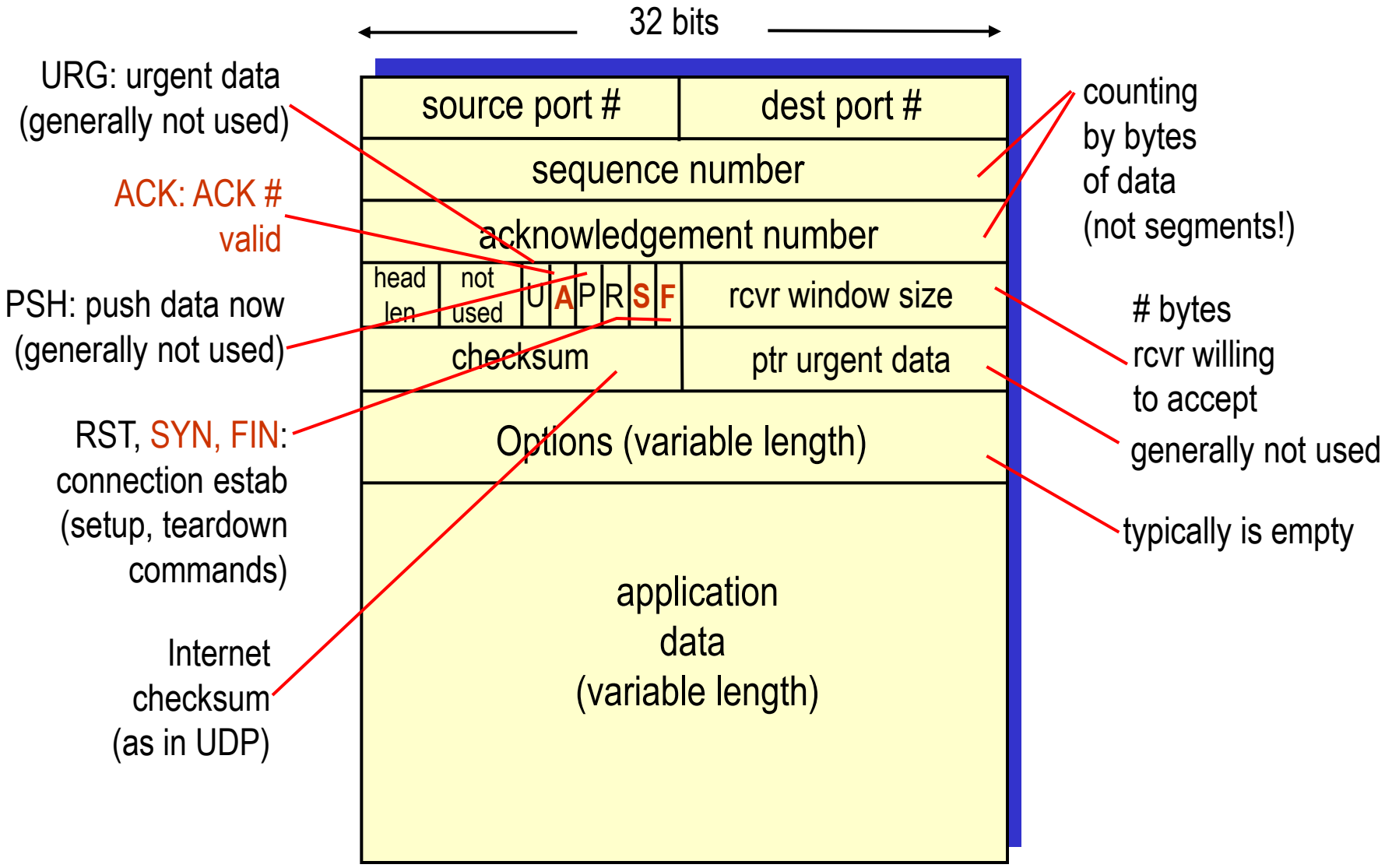
```
Socket clientSocket = new Socket("hostname","80");
```

- *server:* contacted by client

```
ServerSocket welcomeSocket = new ServerSocket(80);

connectionSocket = welcomeSocket.accept();
```

TCP segment structure



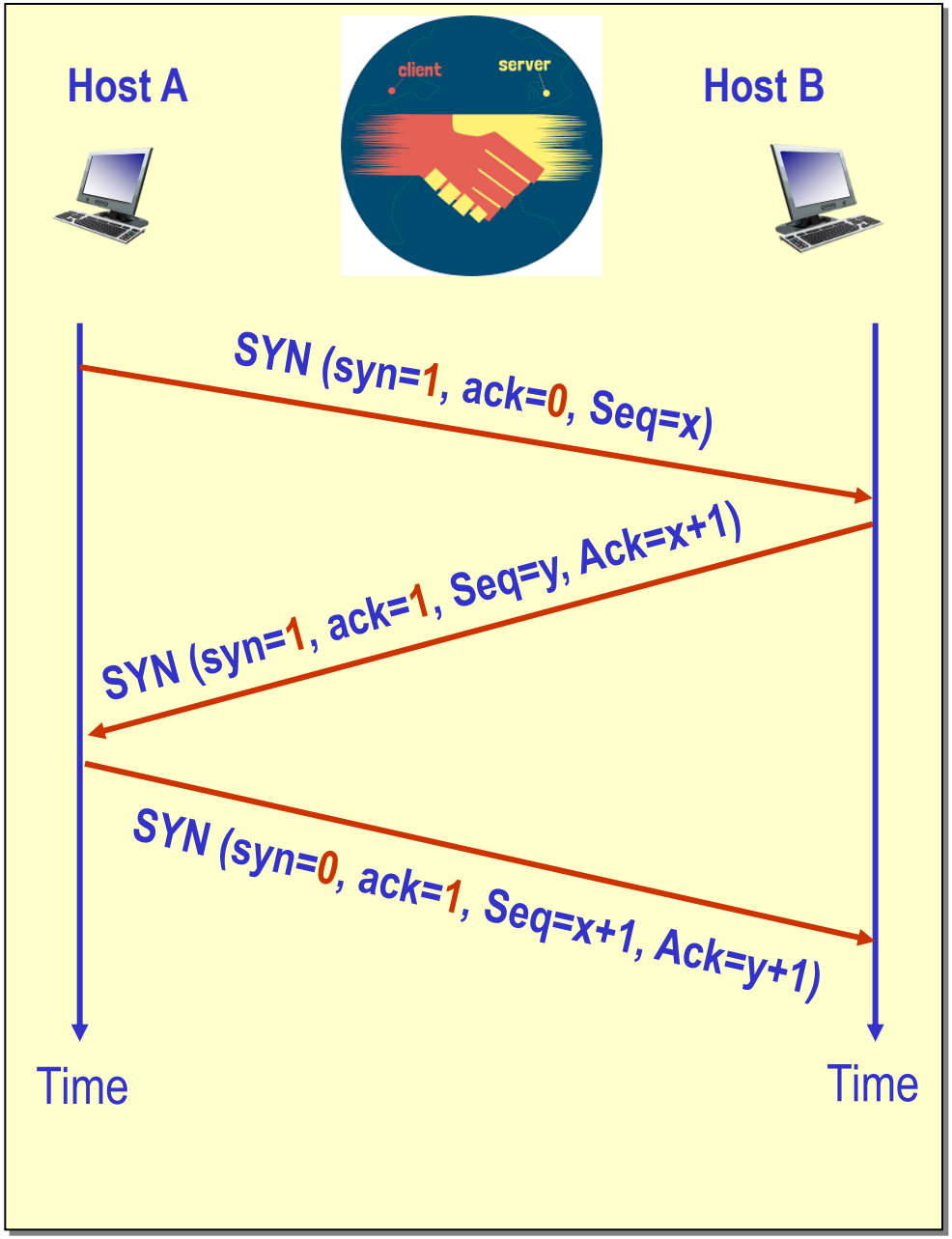
TCP Connection Management

Three way handshake:

Step 1: client end system sends TCP SYN control segment to server
- specifies initial seq #
- No data

Step 2: server end system receives SYN, replies with SYNACK control segment
- ACKs received SYN
- allocates buffers
- specifies server-> receiver initial seq. #

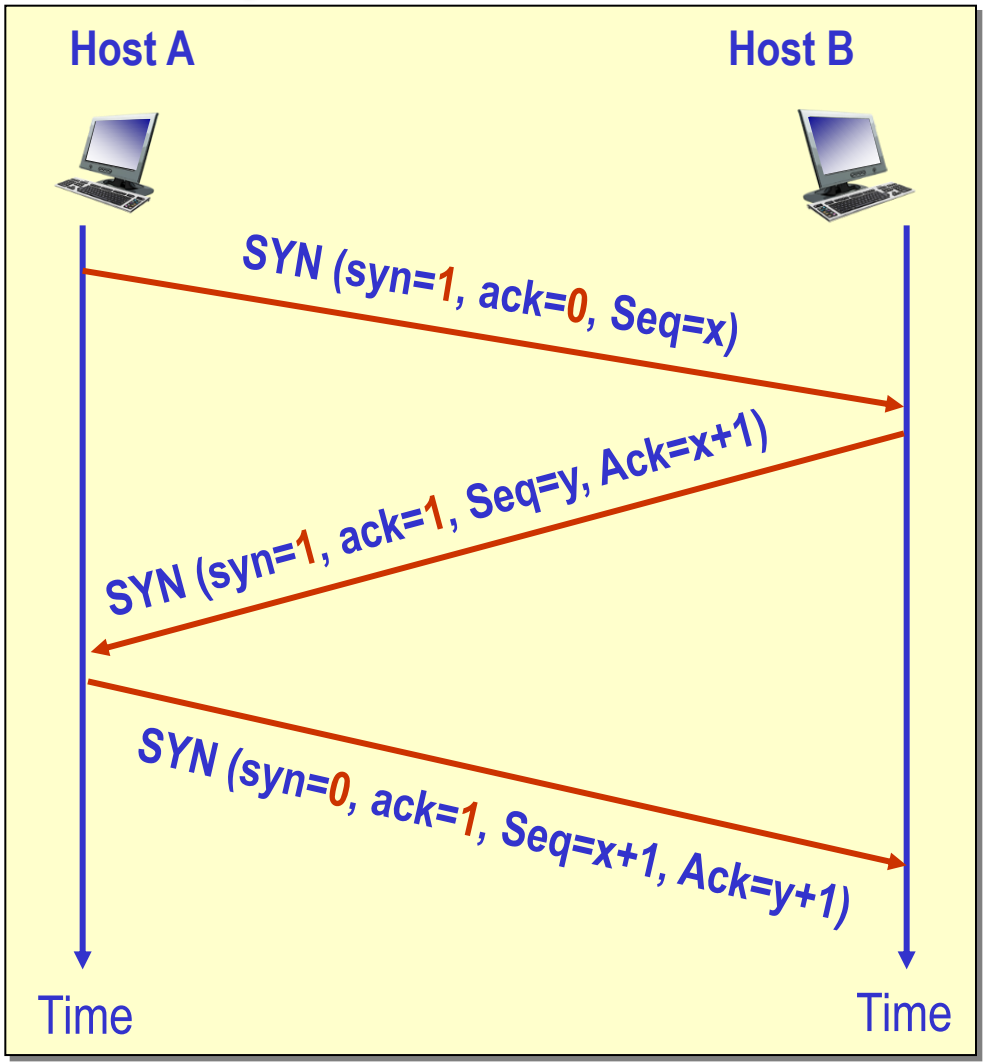
Step 3: client end system sends final ack control segment to server
- The SYN bit is set to 0
- May contain data



TCP Connection Management

Why random sequence numbers?

- To reduce the likelihood that the old packet is viewed as part of the ongoing transfer, the starting sequence number changes over time.
- Because of security issues like connection spoofing by somebody on a different network (see RFC 6528). Sequence Number Attack.

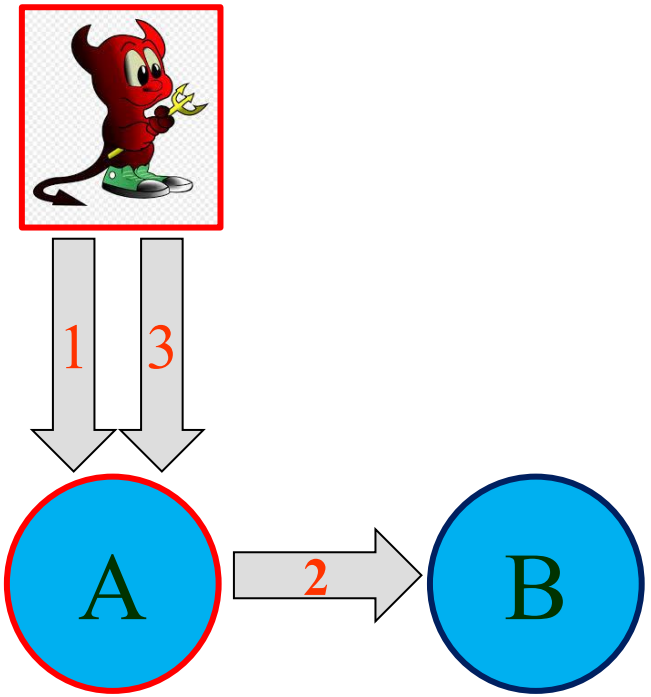


https://www.coursera.org/lecture/real-time-cyber-threat-detection/tcp-sequence-number-attack-CPclp (5:36 min)

TCP Connection Management

Why random sequence numbers?

- To reduce the likelihood that the old packet is viewed as part of the ongoing transfer, the starting sequence number changes over time.
- Because of security issues like connection spoofing by somebody on a different network (see RFC 6528). Sequence Number Attack.

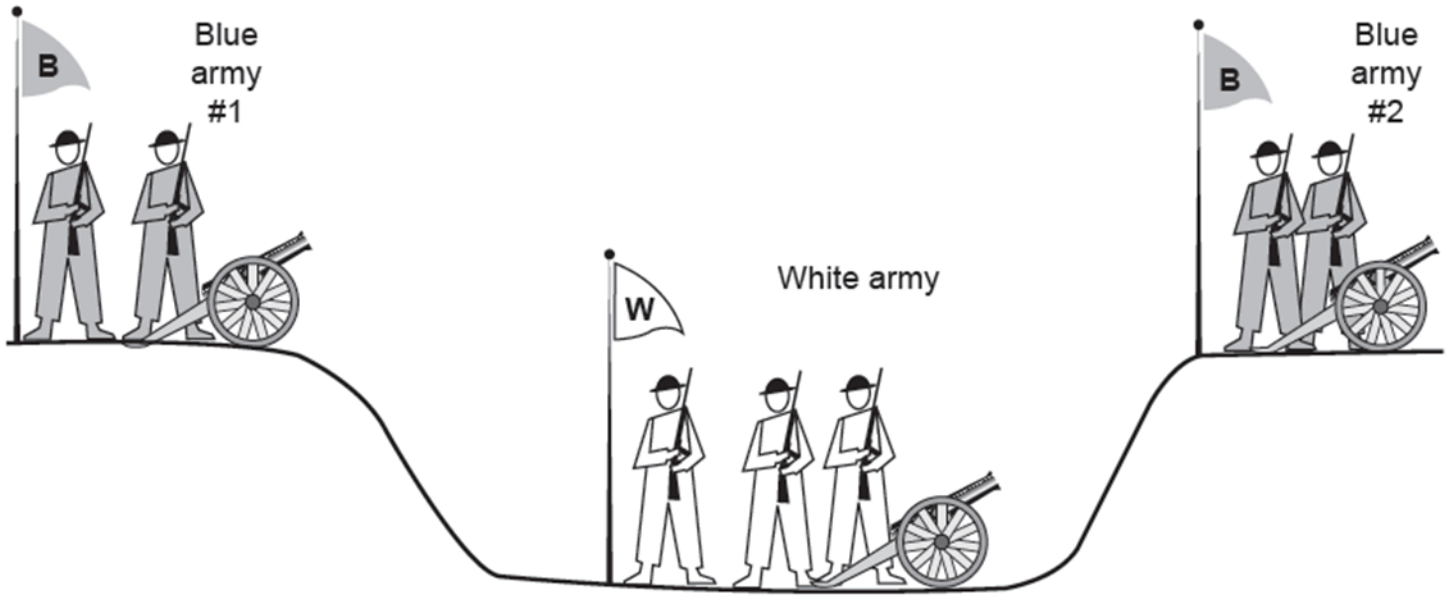


Sequence Number Attack

The coordinated attack problem

בעיית ההתקפה המתואמת

(Closing a TCP connection)



TCP Connection Management (cont.)

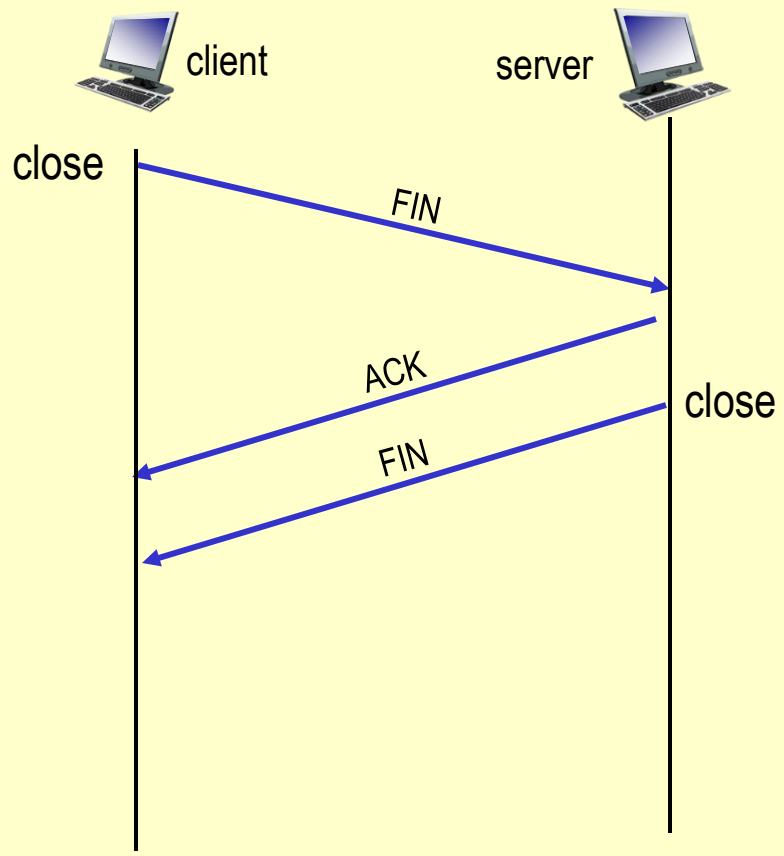
Closing a connection:

client closes socket:

clientSocket.close();

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN.

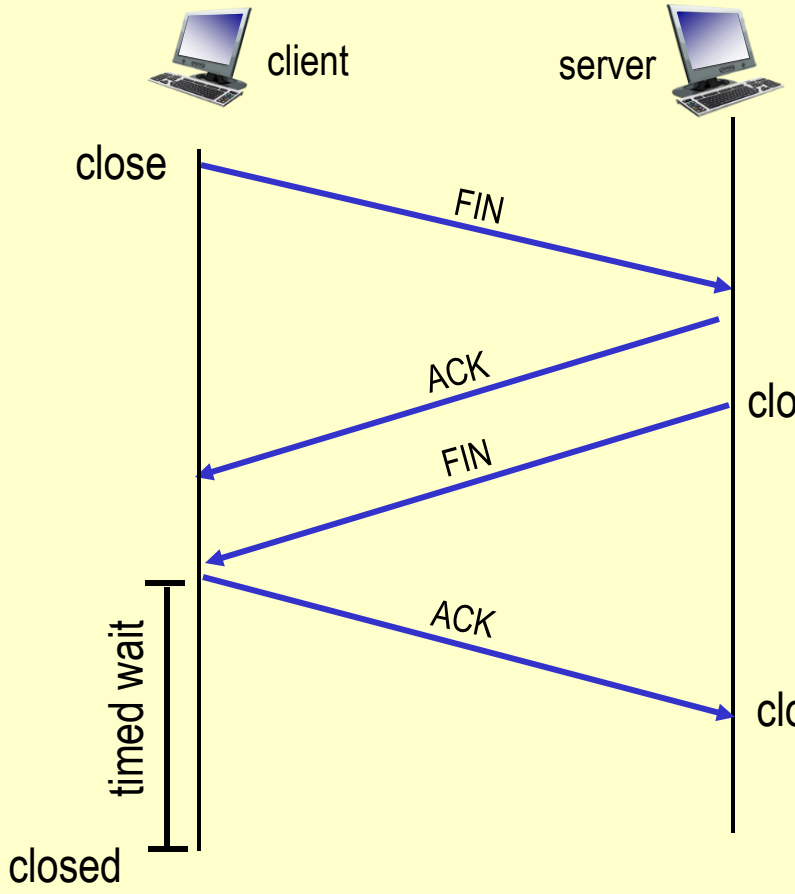


TCP Connection Management (cont.)

Step 3: client receives FIN, replies with ACK.

- enters "timed wait" - will respond with ACK to received FINs
- typically 30 seconds

Step 4: server, receives ACK. Connection closed.



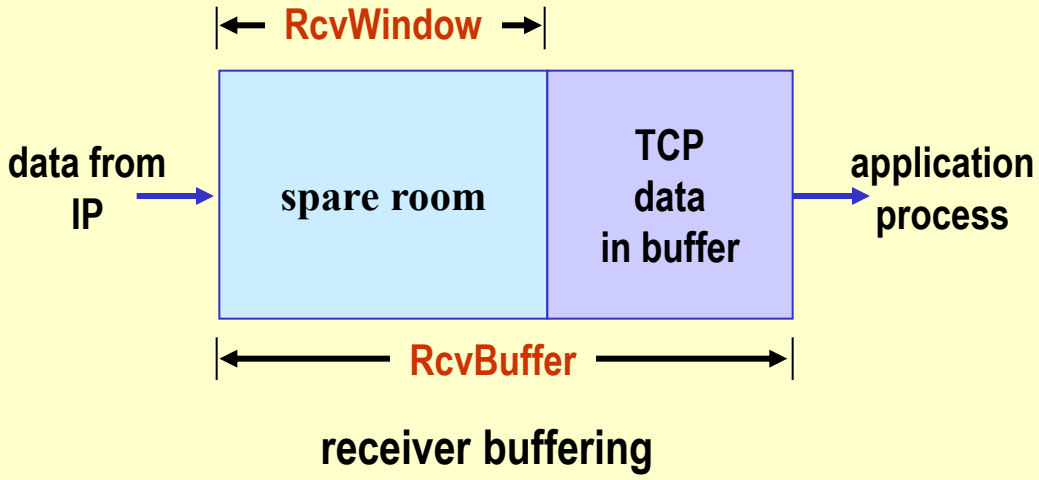
Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - ✓ Reorder
  - ✓ TCP – The Real thing
  - ✓ TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)

TCP Flow Control

Sender won't overrun receiver's buffers by transmitting too much, too fast

`RcvBuffer` = size or TCP Receive Buffer
`RcvWindow` = amount of spare room in Buffer



receiver buffering

Receiver: explicitly informs sender of (dynamically changing) amount of free buffer space

- **`RcvWindow` field** in TCP segment

Sender: keeps the amount of transmitted, unACKed data less than most recently received **`RcvWindow`**

Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - ✓ Reorder
  - ✓ TCP – The Real thing
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)

TCP Congestion Control

Congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- different from flow control!
- manifestations:
  - lost packets (buffer overflow at routers)
  - long delays (queuing in router buffers)

How does the sender perceive congestion?

- loss event = timeout *or* 3 duplicate acks

Reduce Window size:

- TCP sender reduces rate (**`CongWin`**) after loss event

Increase Widow size:

Two Phases

- slow start
- congestion avoidance – AIMD

Important variables:

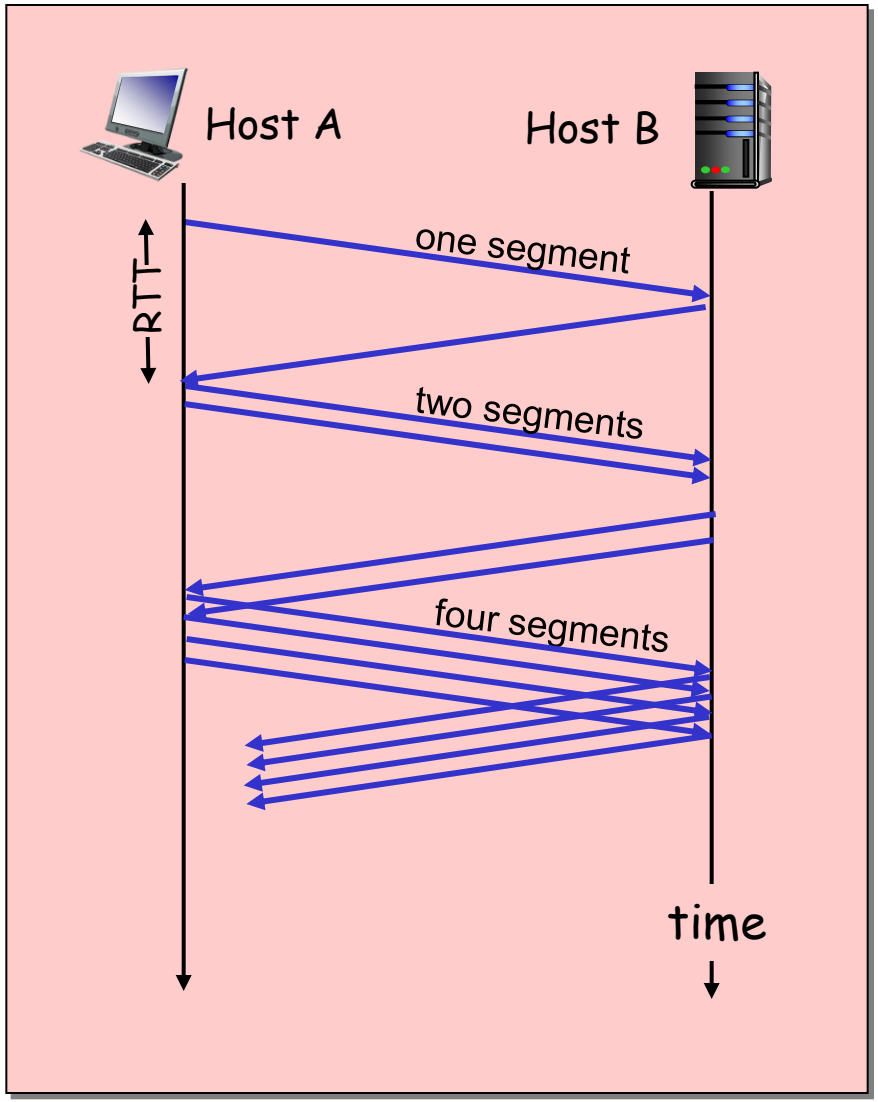
- **`Congwin`**
- **`threshold`**

TCP Slowstart

Slowstart algorithm

initialize: Congwin = 1
for (each segment ACKed)
    Congwin++
until (loss event OR
    CongWin > threshold)

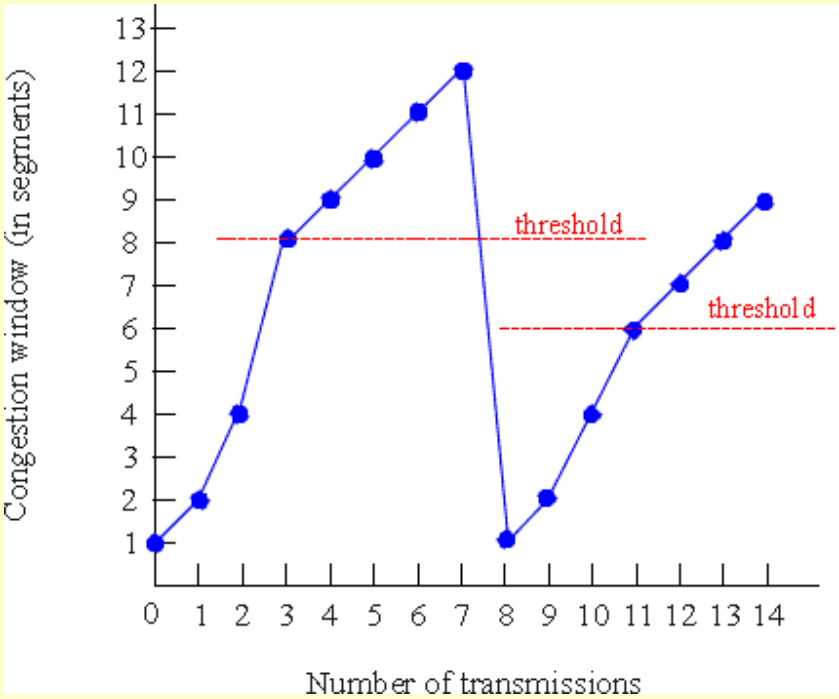
- initial rate is slow but ramps up exponentially fast
- loss event: timeout (Tahoe TCP) and/or three duplicate ACKs (Reno TCP)



TCP Congestion Avoidance

Congestion avoidance

```
/* slowstart is over        */
/* Congwin > threshold */
Until (loss event) {
   every w segments ACKed:
       Congwin++
   }
threshold = Congwin/2
Congwin = 1
perform slowstart[1]
```



TCP Reno skips slowstart (fast recovery) after three duplicate ACKs

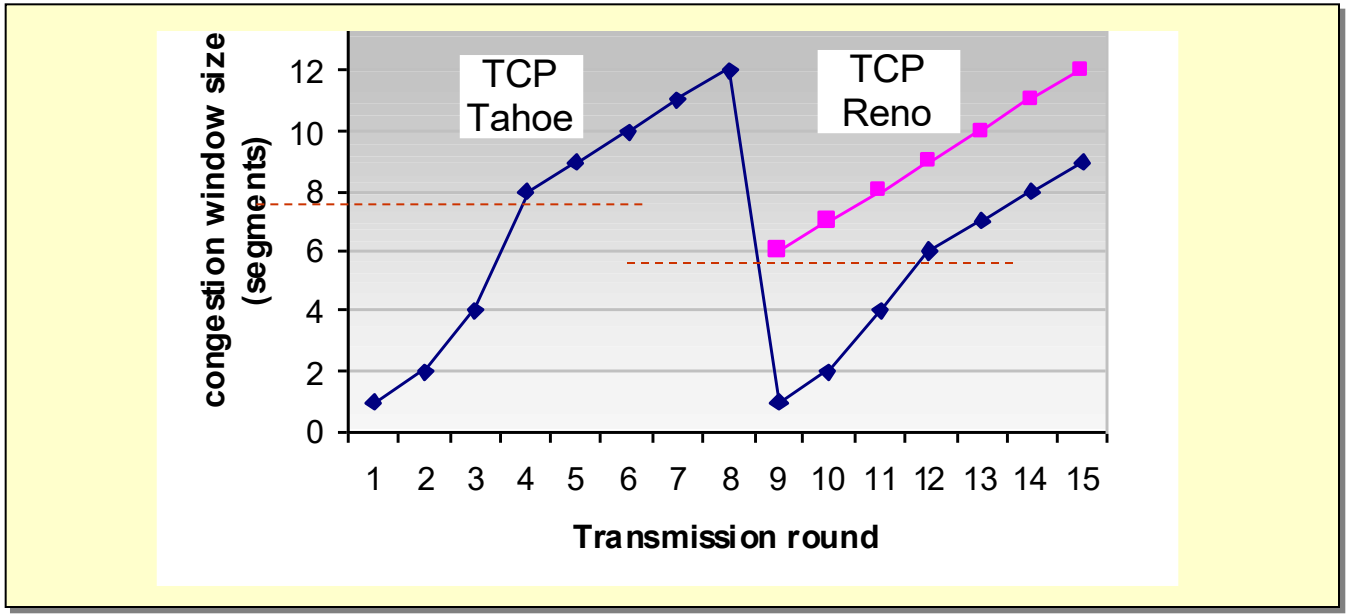
Refinement

- After 3 dup ACKs:
  - **`threshold`** is set to CongWin/2
  - **`CongWin`** is cut **in half**
  - window then grows linearly
- But after timeout event:
  - **`threshold`** is set to CongWin/2
  - **`CongWin`** is set to **1** MSS;
  - window then grows exponentially
  - to a threshold, then grows linearly

Philosophy

- 3 dup ACKs indicates network capable of delivering some segments
- timeout before 3 dup ACKs is "more alarming"

3 dup ACKS Tahoe vs. Reno



- TCP Tahoe – early version
  - unconditionally cuts congestion window to 1.
- TCP Reno – newer version
  - cancels the slow-start phase after a triple duplicate ACK.

The threshold initial value

Most modern TCP implementations often set the threshold to *infinite* (meaning a very high value like 2^31). Using an infinite value means that the congestion window will grow exponentially until a loss occurs. This way the optimal value is found more quickly.

Summary: TCP Congestion Control

- When **`CongWin`** is below **`Threshold`**, sender in slow-start phase, window grows exponentially.
- When **`CongWin`** is above **`Threshold`**, sender is in congestion-avoidance phase, window grows linearly.
- When a triple duplicate ACK occurs, **`Threshold`** set to **`CongWin/2`** and **`CongWin`** set to **`Threshold`**.
- When timeout occurs, **`Threshold`** set to **`CongWin/2`** and **`CongWin`** is set to 1 MSS.

Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - ✓ Reorder
  - ✓ TCP – The Real thing
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)

Problems with TCP

- Slow start
  - Small files doesn't get to full speed
- Head of line blocking
  - Won't deliver content after a lost packet
- Wireless loss does not imply congestion
  - Good for wired, less for wireless
- Slow handshake
  - Take a lot of time to establish a connection

https://www.youtube.com/watch?v=ejcESQOUQV8
(6:03 min)

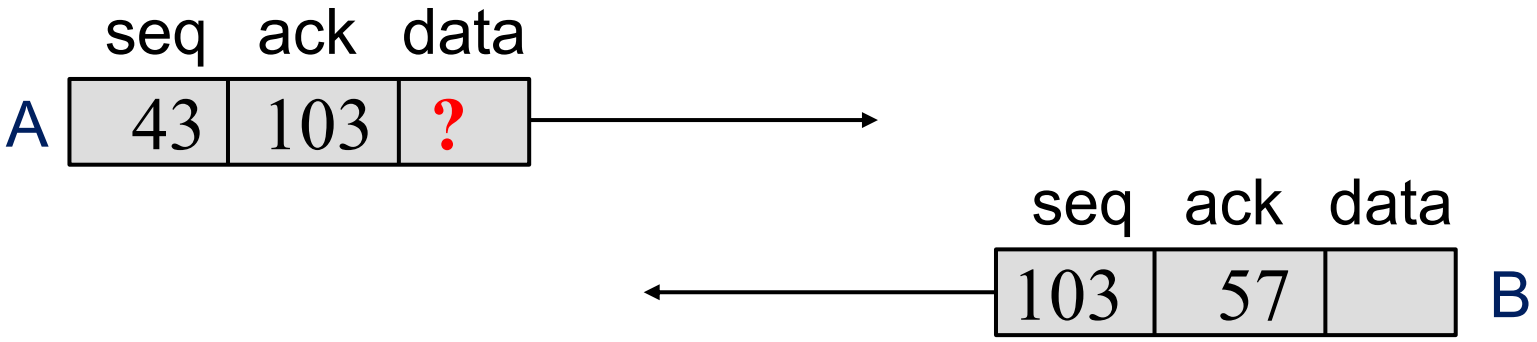
Summary: Transport layer

- Principles behind transport layer services:
  - multiplexing/demultiplexing
  - reliable data transfer
  - flow control
  - congestion control
- Instantiation and implementation in the Internet
  - UDP
  - TCP
- Next lecture: leaving the network "edge" (application, transport layer) into the network "core"

שאלה

מחשב A שולח סיגמנט TCP למחשב B, בו המספר הסידורי הוא 43 וה- ACK הוא 103, ומחשב B שולח בתגובה סיגמנט בו המספר הסידורי הוא 103 וה- ACK הוא 57. כמה בתים של נתונים נשלחו בסיגמנט הראשון ?

Host A sends a TCP segment to host B, in which the sequence number is 43 and the ACK number is 103. Host B sends a segment in response with sequence number 103 and ACK number 57. How many bytes of data were sent in the first segment?



Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

במנגנון ה- congestion control ב- TCP Reno, כאשר ה- timer מצלצל אצל השולח, גודל threshold נקבע למחצית מערכו.

1. נכון
2. לא נכון

In TCP Reno congestion control mechanism, when the timer invokes an event at the sender's side, the threshold value is set to half its current value.

1. True
2. False

Question from an exam
Circle the correct answer

שאלה מבחינה (5 נקודות)
הקף בעיגול את התשובה המתאימה

השדה ב- TCP הנקרא RcvWindow (ז״א חלון הקבלה) משמש ל:

1. בקרת זרימה בלבד.
2. בקרת עומס בלבד.
3. בקרת זרימה וגם בקרת עומס.
4. גילוי שגיאות.
5. כל התשובות (1 עד 4) לא נכונות.

The RcvWindow field in TCP uses for:

1. Flow control only
2. Congestion control only
3. Flow control and congestion control
4. Error detection
5. Answers 1-4 are all false

Question from an exam
Circle the correct answer

שאלה מבחינה (5 נקודות)
הקף בעיגול את התשובה המתאימה

בשלב ה- slow start ב- TCP מתקיים:

1. גידול החלון איטי (לכל היותר ב- 1 במהלך RTT).
2. גידול החלון מהיר (החלון יכול להכפיל את גודלו במהלך RTT).
3. שלב זה אופציונאלי ותלוי בבררת המחדל של המשתמש.
4. מוקם קשר TCP עם חלון בגודל קבוע.
5. כל התשובות (1 עד 4) לא נכונות .

During slow start in TCP:

1. Window grows slowly (at most 1 each RTT)
2. Window grows fast (may double its size each RTT)
3. This phase is optional and determined by the user's set defaults
4. A TCP connection is established with a constant size window
5. Answers 1-4 are all false

Today

- Transport layer
  - ✓ Concepts of Reliable Data Transfer
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - ✓ Reorder
  - ✓ TCP – The Real thing
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)